QAT

Joe Boudreau 2016

September 14, 2020

Contents

1 Introduction

The `QAT` system (pronouced "cat") is comprised of packages: `QatGenericFunctions` `QatDataAnalysis`, `QatDataModelling`, `QatPlotting`, `QatPlotWidgets`, `QatProject`, and the optional addon `QatInventorWidgets`. QAT is a toolkit based upon the `Qt` system, which is used throughout the toolkit for platform-independent builds, and in parts of the toolkit for plotting.

These are designed to enable function arithmetic, integrating, solution of differential equations, solution of problems in classical mechanics, plotting of functions and data, data analysis and data modeling. I have assembled this package to enable solution to a number of computational problems that arise in graduate physics and engineering, the idea being to fill gaps that are not filled by more standard and widely available packages. The normal order of preference is as follows. Where possible, one uses software from the C++ standard library. Where that fails, one uses widely available software e.g. `Boost`. After that, one considers other options, free or otherwise. And finally, if all else fails–write the software yourself.

This taxonomy is nuanced, because other considerations arise, namely, a software package should be available in the right language, with and acceptable interface, possibly requiring thread safety, or a certain level of maintenance, and it may drag in unwanted dependencies, including dependencies on packages which are commercial or which have restrictive licenses. Ultimately whether a package is usable is a matter of personal choice, except if the choice is taken by an Organization, in which case it is taken by the Big Bosses.

`QAT` therefore uses the Gnu Scientific library (`gsl`) for the implementation of special functions; the `Qt` Toolkit for graphical user interfaces, the superb `Eigen` package for matrix manipulation and solution of problems in linear algebra, the `HDF5` and/or `Root` libraries for machine dependent input and output, and `MINUIT` for function minimization. These classes are mostly hidden from the user–except for Eigen, which we strongly recommend learning and using in parallel with `QAT`. For users who are adventurous and so inclined, a mastery of the `Qt` library will be rewarding, since, together with `QatPlotWidgets`, it will allows decent plotting capabilities to be combined with rich graphical uses interfaces, all of which can be quickly designed using the `QtCreator` or `QtDesigner` products.

Therefore a basic working toolkit consists of the packages:

- `QAT` (`QatGenericFunctions`, `QatDataAnalysis`, `QatDataModeling`, `QatPlotting`, `QatPlotWidgets`, `QatProject`)
- `Qt`
- `Eigen`
- `HDF5`

It is often highly useful to visualize calculations in three dimensions. For this purpose we have bundled

- The `Coin` implementation of the Open Inventor library
- The `SoQt` package

- `QatInventorWidgets` library

The first two items on this list are no longer maintained by the orginal developers, which does represent a serious drawback, though in our opinion they represent the best noncommercial solution, so we maintain and distribute them ourselves. As Wikipedia puts it, "Despite its age, the Open Inventor API is still widely used for a wide range of scientific and engineering visualization systems around the world, having proven itself well designed for effective development of complex 3D application software." Commericial versions of both APIs are available from the FEI company, for those willing to pay for such a solution.

1.1 Installation instructions

Always up-to-date installation procedures for MacIntosh and for Linux systems can be found at the website `qat.pitt.edu`. Support for additional platforms is a future development goal.

1.2 Building programs

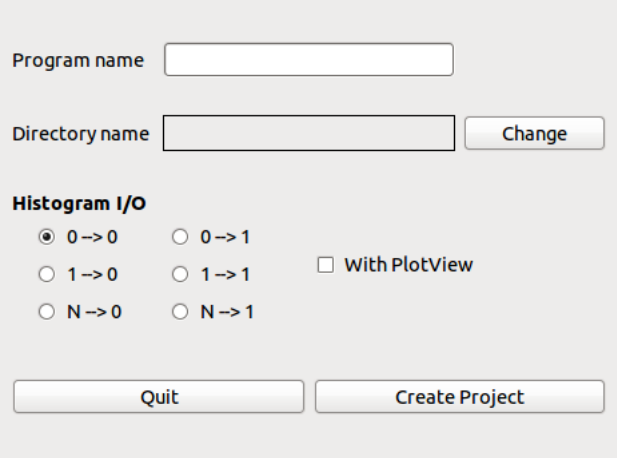
The build of `Qat`, as well as the build of user programs linking agains `Qat`, uses the multiplatform application system `Qt`, and especially the qmake procedure. `Qmake`, starting with a project file, creates a makefile which is platform-specific. The `Qat` system provides a tool to interactively generate:

- A directory structure: `PROGRAM/src`, `PROGRAM/local`, `PROGRAM/local/bin` where `PROGRAM` is the name of a program
- Source code `PROGRAM/src/program.cpp`
- Project file `PROGRAM/src/qt.pro`

To make an almost-empty program, type from the command line

- `qat-project`

This will conjure a dialog that looks like



which creates all of the above. It can also automatically include the code for histogram file input and output with options specified on your program's command line, and/or interactive plotting. When you have created your project, you can build it by typing

- `cd PROGRAM/src`
- `qmake`
- `make`

The new program lives in `PROGRAM/local/bin/program`. In case unresolved symbols are reported at runtime, you may need to:

- `export LD_LIBRARY_PATH=/usr/local/lib`.

2 QatGenericFunctions Reference Manual

The Qat API is described in the following sections. *All of the classes described in this section are scoped within the namespace* `Genfun`.

2.1 QatGenericFunctions: Kernel Classes

The `QatGenericFunction` library contains functors implementing functions of one or more variable. Core functionality is provided by the class `AbsFunction, AbsFunctional, AbsParameter, Parameter, Argument, and ArgumentList.` In addition to that, a number of classes such as `FunctionSum`, `FunctionDifference`, `FunctionDirectProduct`, etc., which are necessary for the basic functionality of the library, are "internal" and are not described here.

2.1.1 class AbsFunction

`#include "QatGenericFunctions/AbsFunction.h"`

Description AbsFunction is the base class for all function objects. The functions they represent may take one or more variable as an argument; in case of functions of more than one variable, the class `Argument` is used to aglomerate variables into a single structure. The four basic operations (+,-,*,/) are defined for all `AbsFunction`s, and operands may be either

- two `AbsFunction`s
- an `AbsFunction` and a `double` (in either order)
- an `AbsParameter` and an `AbsFunction` (in either order).

`AbsFunctions` can be composed with each other; eg. `g=f(h)` where `f`,`g`, and h are `AbsFunctions`. Also, the direct product operator (%) forms the direct product (f%g) of two `AbsFunction`s and therefore can be used to build multivariate functions from univariate functions. The result of such operations is most convieniently typed as `GENFUNCTION`, typedef'd back to `const AbsFunction &`.

The `QatGenericFunction` library is endowed with an automatic derivative system, which allows to obtain the derivative (partial derivative for functions of more than one variable).

Type Definitions

- `typedef const AbsFunction & GENFUNCTION`

Methods Default constructor

- `AbsFunction()`

Copy constructor

- `AbsFunction(const AbsFunction &right)`

Destructor

- `virtual ~AbsFunction()`

Dimensionality

- `virtual unsigned int dimensionality() const`

Function value

- `virtual double operator() (double argument) const`
- `virtual double operator() (const Argument &argument) const`

Clone

- `virtual AbsFunction * clone() const`

Function composition. The return value (`FunctionComposition`) is a type of `AbsFunction`.

- `virtual FunctionComposition operator () (const AbsFunction &f) const`

Parameter composition. The return value (`ParameterComposition`) is a type of AbsParameter (see below).

- `virtual ParameterComposition operator() ( const AbsParameter &p) const`

Derivatives. The return value (`Derivative`) is a type of `AbsFunction`.

- `Derivative partial(const Variable &v) const`
- `Derivative prime() const`
- `Derivative partial(unsigned int) const`

Test for analytic derivative

- `virtual bool hasAnalyticDerivative() const`

Functions The following list of functions implements the function algebra described above. The return types for all of these functions are all a type of `AbsFunction`.

- `FunctionProduct operator * (const AbsFunction &op1, const AbsFunction &op2)`
- `FunctionSum operator + (const AbsFunction &op1, const AbsFunction &op2)`
- `FunctionDifference operator - (const AbsFunction &op1, const AbsFunction &op2)`
- `FunctionQuotient operator / (const AbsFunction &op1, const AbsFunction &op2)`
- `FunctionNegation operator - (const AbsFunction &op1)`
- `ConstTimesFunction operator * (double c, const AbsFunction &op2)`
- `ConstPlusFunction operator + (double c, const AbsFunction &op2)`
- `ConstMinusFunction operator - (double c, const AbsFunction &op2)`
- `ConstOverFunction operator / (double c, const AbsFunction &op2)`
- `ConstTimesFunction operator * (const AbsFunction &op2, double c)`

- `ConstPlusFunction operator + (const AbsFunction &op2, double c)`
- `ConstPlusFunction operator - (const AbsFunction &op2, double c)`
- `ConstTimesFunction operator / (const AbsFunction &op2, double c)`
- `FunctionTimesParameter operator * (const AbsFunction &op1, const AbsParameter &op2)`
- `FunctionPlusParameter operator + (const AbsFunction &op1, const AbsParameter &op2)`
- `FunctionPlusParameter operator - (const AbsFunction &op1, const AbsParameter &op2)`
- `FunctionTimesParameter operator / (const AbsFunction &op1, const AbsParameter &op2)`
- `FunctionTimesParameter operator * (const AbsParameter &op1, const AbsFunction &op2)`
- `FunctionPlusParameter operator + (const AbsParameter &op1, const AbsFunction &op2)`
- `FunctionPlusParameter operator - (const AbsParameter &op1, const AbsFunction &op2)`
- `FunctionTimesParameter operator / (const AbsParameter &op1, const AbsFunction &op2)`
- `FunctionDirectProduct operator % (const AbsFunction &op1, const AbsFunction &op2)`

2.1.2 class AbsParameter

`#include "QatGenericFunctions/AbsParameter.h"`

Description `AbsParameter` is a base class for parameters. A parameter is a class with a directly or indirectly adjustable value. It may appear in expressions with other AbsParameters; then when one of the `AbsParameter` values changes the expression is automatically updated. `AbsParameters` also can be used in expressions with functions. When a parameter changes value, the expression changes shape. See also `Parameter`.

Typedefs typedef const AbsParameter & GENPARAMETER

Methods Constructor:

- `AbsParameter();`

Copy Constructor:

- `AbsParameter(const AbsParameter &);`

Destructor:

- `virtual ~AbsParameter();`

Clone:

- `virtual AbsParameter * clone() const;`

Get parameter value:

- `virtual double getValue() const;`

Functions The following list of functions implements the function algebra described above. The return types for all of these functions are all a type of AbsParameter.

- `ConstTimesParameter operator * (double c, const AbsParameter &op2) ;`
- `ConstPlusParameter operator + (double c, const AbsParameter &op2) ;`
- `ConstMinusParameter operator - (double c, const AbsParameter &op2) ;`
- `ConstOverParameter operator / (double c, const AbsParameter &op2) ;`
- `ConstTimesParameter operator * (const AbsParameter &op2, double c);`
- `ConstPlusParameter operator + (const AbsParameter &op2, double c);`
- `ConstPlusParameter operator - (const AbsParameter &op2, double c);`
- `ConstTimesParameter operator / (const AbsParameter &op2, double c);`
- `ParameterProduct operator * (const AbsParameter &op1, const Abs Parameter &op2);`
- `ParameterSum operator + (const AbsParameter &op1, const Abs Parameter &op2);`
- `ParameterDifference operator - (const AbsParameter &op1, const Abs Parameter &op2);`
- `ParameterQuotient operator / (const AbsParameter &op1, const Abs Parameter &op2);`
- `ParameterNegation operator - (const AbsParameter &op1);`

2.1.3 class AbsFunctional

`#include "QatGenericFunctions/AbsFunctional.h"`

Description AbsFunctional is a base class for functionals, i.e. objects that act upon a function to obtain a real number.

Methods Constructor

- `AbsFunctional();`

Destructor:

- `virtual ~AbsFunctional();`

Function call operator:

- `virtual double operator() (const AbsFunction & function) const;`

2.1.4 class Argument

`#include "QatGenericFunctions/Argument.h"`

Description Argument represents the argument to a function, agglomerating one or more variables. For example, a function of two variables expects a two-element argument.

Methods Constructor

- `Argument(int ndim=0);`

Initializer list constructor:

- `Argument(std::initializer_list<double>);`

Copy Constructor

- `Argument( const Argument &);`

Assignment operator

- `const Argument & operator=(const Argument &);`

Destructor:

- `~Argument();`

Set/Get Value

- `double & operator[] (int i);`
- `const double & operator[] (int i) const;`

Get the dimensions

- `unsigned int dimension() const;`

2.1.5 typedef ArgumentList

`#include "QatGenericFunctions/ArgumentList.h"`

Description The class `ArgumentList` is an `std::vector` of `Arguments`:

- `typedef std::vector<Argument> ArgumentList`

2.1.6 template<class Type> class Cut

`#include "QatGenericFunctions/CutBase.h"`

Description `Cut` is a template class, parameterized on a class `Type`, the type of thing upon which one desires to cut. Its function call operator evaluates to `true` or `false`. Uses of this class should instantiate the class on a concrete datatype, and then derive subclasses implementing actual cuts. The advantage is that the derived subclasses obey a boolean algebra, i.e. respond to the operators ||, &&, and !.

Methods Constructor:

- `Cut();`

Copy Constructor:

- `Cut(const Cut & right);`

Destructor

- `virtual ~Cut();`

Clones the cut:

- `virtual Cut * clone() const;`

Function call operator:

- `virtual bool operator ()( const Type & t ) const;`

OR operation (return value is a Cut<Type>):

- OR operator ||(const Cut<Type> & A) const;

AND operation (return value is a Cut<Type>):

- `AND operator &&( const Cut<Type> & A ) const;`

NOT operation (return value is a Cut<Type>):

- `NOT operator !  ( void ) const;`

2.1.7 class Parameter

`#include "QatGenericFunctions/Parameter.h"`

Inherits `AbsParameter`

Description This class is a simple low-level double precision number, together with some limiting values. It is designed essentially as an ingredient for building function objects. Parameters can be connnected to one another and combined in algebraic expressions with constants and with other parameters. If a parameter is connected, it takes is value (and limits) from the parameter to which it is connected. It can be disconnected by connecting to NULL. When disconnected, it captures the values of the connected field before dropping the connection. An attempt to alter the values of the field while the `Parameter` is connected to another parameter will result in an obnoxious warning mesage.

Methods Constructor:

- `Parameter(std::string name, double value, double lowerLimit=-1e100, double upperLimit= 1e100);`

Copy constructor

- `Parameter(const Parameter & right);`

Destructor

- virtual ~Parameter();

Assignment

- `const Parameter & operator=(const Parameter &right);`

Accessor for the Parameter name

- `const std::string & getName() const;`

Accessor for value

- `virtual double getValue() const;`

Accessor for Lower Limit

- `double getLowerLimit() const;`

Accessor for Upper Limit

- `double getUpperLimit() const;`

Set Value

- `void setValue(double value);`

Set Lower Limit

- `void setLowerLimit(double lowerLimit);`

Set Upper Limit

- `void setUpperLimit(double upperLimit);`

Take values + limits from some other parameter.

- `void connectFrom(const AbsParameter * source);`

Functions

- `std::ostream & operator << ( std::ostream & o, const Parameter &p);`

2.1.8 Enum NormalizationType

`#include "QatGenericFunctions/Defs.h"`

Description The `NormalizationType` is used in the constructor of certain orthogonal polynomials. `STD` means to construct the standard polynomial, i.e. unnormalized, and `TWIDDLE` means to construct normalized orthogonal polynomials.

2.2 QatGenericFunctions: Function Classes

The classes in this section implement specific mathematical functions, like `sin`, `cos`, `exp`, and other basic and transcendental functions, all inheriting from `AbsFunction`. Where possible we have taken the actual computation from the runtime library; when no such function exists we prefer to take the implementation from the gnu scientific library (GSL); when none can be found we implement our own.

Some functions (e.g. `AssociatedLegendrePolynomial`) have constructors taking arguments (like the order and degree of the polynomial); others may have internal parameters that affect the shape of the function. These parameters may be set to the desired value or connected to other parameters as desired.

2.2.1 class Abs

`#include "QatGenericFunctions/Abs.h"`

Description: Takes the absolute value of the argument.

Implementation: Calls `std::abs`

Dimensionality: Function of one variable.

Methods: Constructor:

- `Abs();`

2.2.2 class Airy

`#include "QatGenericFunctions/Airy.h"`

Description: Implements the Airy functions $\mathrm{Ai}(x)$ and $\mathrm{Bi}(x)$.

Enums: `enum Type {FirstKind,SecondKind}`

Implementation: Calls `gsl_sf_airy_ai_e` or `gsl_sf_airy_bi_e` from the gnu scientific library.

Dimensionality: Function of one variable.

Methods: Constructor. Type may be `Airy::Ai` or `Airy::Bi`.

- `Airy(Airy::Type type);`

2.2.3 class ACos

`#include "QatGenericFunctions/ACos.h"`

Description: Takes the arc cosine of its argument.

Implementation: Calls `std::acos`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `ACos();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.4 class ACosh

`#include "QatGenericFunctions/ACosh.h"`

Description: Takes the inverse hyperbolic cosine of its argument.

Implementation: Calls `std::acosh`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `ACosh();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.5 class ArrayFunction

`#include "QatGenericFunctions/ArrayFunction.h"`

Description: The ArrayFunction takes its values from an array, which it copies in.

Implementation: Native implementation.

Dimensionality: Function of one variable.

Methods: Constructor:

- `ArrayFunction(const double *begin, const double *end);`

Initializer list constructor:

- `ArrayFunction(std::initializer_list<double>);`

2.2.6 class ASin

`#include "QatGenericFunctions/ASin.h"`

Description: Takes the arc sine of its argument.

Implementation: Calls `std::asin`

Dimensionality: Function of one variable

Methods: Constructor:

- `ASin();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.7 class ASinh

`#include "QatGenericFunctions/ASinh.h"`

Description: Takes the inverse hyperbolic sine of its argument.

Implementation: Calls `std::asinh`

Dimensionality: Function of one variable

Methods: Constructor:

- `ASinh();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.8 class AssociatedLaguerrePolynomial

`#include "QatGenericFunctions/AssociatedLaguerrePolynomial.h"`

Description: Implements the associated Laguerre polynomial, $L_n{}^{\alpha}(x)$ of degree n and constant α. These polynomials are orthogonal and with the standard normalization the orthogonality relation is

$$\int_0^\infty L_n^\alpha(x) L_m^\alpha(x) x^\alpha e^{-x} dx = \frac{\Gamma(n+\alpha+1)}{n!}\delta_{nm}$$

Normalized associated Laguerre polynomials are denoted as $\tilde{L}_n^\alpha(x)$ and are both orthogonal and normal. They can be obtained by specifying `TWIDDLE` normalization to the constructor.

Implementation: Calls the function `gsl_sf_laguerre_n_e` from the gnu scientific library (GSL).

Dimensionality: Function of one variable.

Methods: Constructor taking the degree n and the constant a. Normalization type specifies non-normalized (`STD`) or normalized (`TWIDDLE`) orthogonal polynomials.

- `AssociatedLaguerrePolynomial(unsigned int n, double a, NormalizationType type=STD);`

Get the degree n:

- `unsigned int n() const;`

Get the constant a:

- `double a() const;`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.9 class AssociatedLegendre

`#include "QatGenericFunctions/AssociatedLegendre.h"`

Description: Implements the associated Legendre function, $\mathrm{P_l{}^m}(x)$ of degree l and order m. These functions are orthogonal with the standard normalization the orthogonality relation:

$$\int_{-1}^{1} P_l^m(x) P_{l'}^m(x) dx = \frac{2(l+m)!}{(2l+1)(l-m)!}\delta_{ll'}$$

Normalized associated Legendre functions are denoted as $\tilde{P}_l^m(x)$ and are both orthogonal and normal. They can be obtained by specifying `TWIDDLE` normalization to the constructor.

Implementation: Calls the function `gsl_sf_legendre_Plm_e` or `gsl_sf_legendre_sphPlm_e` from the gnu scientific library (GSL), according to whether the polynomial is normalized (`TWIDDLE`) or nonnormalized (`STD`).

Dimensionality: Function of one variable.

Methods: Constructor taking the degree l and the order m. Normalization type specifies non-normalized (`STD`) or normalized (`TWIDDLE`) orthogonal polynomials.

- `AssociatedLegendre(unsigned int n, int m, NormalizationType type=STD);`

Get the degree l:

- `unsigned int l() const;`

Get the order m:

- `int m() const;`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.10 class ATan

`#include "QatGenericFunctions/ATan.h"`

Description: Takes the arc tangent of its argument.

Implementation: Calls `std::atan`

Dimensionality: Function of one variable

Methods: Constructor:

- `ATan();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.11 class ATanh

`#include "QatGenericFunctions/ATanh.h"`

Description: Takes the inverse hyperbolic tangent of its argument.

Implementation: Calls `std::atanh`

Dimensionality: Function of one variable.

Methods: Constructor:

- `ATanh();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.12 class FractionalOrder::Bessel

`#include "QatGenericFunctions/Bessel.h"`

Description: Bessel functions of fractional order. Bessel functions of the first kind and also Bessel functions of the second kind (also called Neumann functions) can be constructed. The order of the Bessel function is an adjustable parameter of class `FractionalOrder::Bessel`.

Implementation: Calls the function `gsl_sf_bessel_Jnu_e` or `gsl_sf_bessel_Ynu_e` from the gnu scientific library.

Dimensionality: Function of one variable.

Enums: `enum Type {FirstKind,SecondKind}`

Methods: Constructor:

- `FractionalOrder::Bessel (Type type);`

Returns the (fractional) order of the Bessel function. Default value is 0.0;

- `Parameter & n();`
- `const Parameter & n() const;`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.13 class IntegralOrder::Bessel

`#include "QatGenericFunctions/Bessel.h"`

Description: Bessel functions of integral order. Bessel functions of the first kind and also Bessel functions of the second kind (also called Neumann functions) can be constructed.

Implementation: Calls the function `gsl_sf_bessel_Jn_e` or `gsl_sf_bessel_Yn_e` from the gnu scientific library.

Dimensionality: Function of one variable

Methods: Constructor. Parameters are the order (n) and the type:

- `Bessel (unsinged int n, Type type);`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.14 class BetaDistribution

`#include "QatGenericFunctions/BetaDistribution.h"`

Description: The Beta distribution is a normalized probability distribution $f(x)$ given by

$$f(x|\alpha,\beta) = \frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{(1-\alpha)}(1-x)^{(1-\beta)}$$

It is parametrized by the constants α and β.

Implementation: Native implementation.

Dimensionality: Function of one variable.

Methods: Constructor:

- `BetaDistribution();`

Get the parameter alpha. Default value is 1.0.

- `Parameter & alpha();`

Get the parameter beta. Default value is 1.0.

- `Parameter & beta();`

2.2.15 class Cos

`#include "QatGenericFunctions/Cos.h"`

Description: Takes the cosine of its argument.

Implementation: Calls `std::cos`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Cos();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.16 class Cosh

`#include "QatGenericFunctions/Cosh.h"`

Description: Takes the hyperbolic cosine of its argument.

Implementation: Calls `std::cosh`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Cosh();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.17 class CubicSplinePolynomial

`#include "QatGenericFunctions/CubicSplinePolynomial.h"`

Description: This function represents the natural cubic spline interpolating a series of control points. A typical use case is to instantiate the function, then add the required control points, then evaluate the function.

Implementation: Native. Uses the `Eigen` package for matrix operations, internally.

Dimensionality: Function of one variable.

Methods: Constructor

- `CubicSplinePolynomial();`

Add a new point to the set:

- `void addPoint(double x, double y);`

Determine the range of interpolation. This is the minimum and maximum abscissa of all the points added so far.

- `void getRange(double &min, double &max) const;`

2.2.18 class CumulativeChiSquare

#include "QatGenericFunctions/CumulativeChiSquare.h"

Description: Cumulative χ^2 distribution for N degrees of freedom, also called the *probability chi square*.

Implementation: Native; uses `Genfun::IncompleteGamma`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `CumulativeChiSquare(unsigned int nDof);`

Get the number of degrees of freedom (nDof):

- `unsigned int nDof() const;`

2.2.19 class EllipticIntegral::FirstKind

`#include "QatGenericFunctions/EllipticIntegral.h"`

Description: Elliptic integral of the first kind

$$f(\phi|k) = \int_0^{\phi} \frac{d\theta}{\sqrt{1 - k^2 sin^2\theta}}$$

Implementation: Calls the function `gsl_sf_ellint_F_e` from the gnu scientific library (GSL).

Dimensionality: Function of one variable.

Methods: Constructor:

- `EllipticIntegral::FirstKind ();`

Get the k-parameter. Default value = 1.0.

- `Parameter & k();`
- `const Parameter & k() const;`

2.2.20 class EllipticIntegral::SecondKind

`#include "QatGenericFunctions/EllipticIntegral.h"`

Description: Elliptic integral of the second kind

$$f(\phi|k) = \int_0^{\phi} \sqrt{1 - k^2 sin^2\theta} d\theta$$

Implementation: Calls the function `gsl_sf_ellint_E_e` from the gnu scientific library (GSL).

Dimensionality: Function of one variable.

Methods: Constructor:

- `EllipticIntegral::SecondKind ();`

Get the k-parameter. Default value = 1.0.

- `Parameter & k();`
- `const Parameter & k() const;`

2.2.21 class EllipticIntegral::ThirdKind

`#include "QatGenericFunctions/EllipticIntegral.h"`

Description: Elliptic integral of the third kind

$$f(\phi|k,n) = \int_0^{\phi} \frac{1}{1 - n\sin^2\theta} \frac{d\theta}{\sqrt{1 - k^2\sin^2\theta}}$$

Implementation: Calls the function `gsl_sf_ellint_P_e` from the gnu scientific library (GSL).

Dimensionality: Function of one variable.

Methods: Constructor:

- `EllipticIntegral::ThirdKind ();`

Get the k-parameter. Default value = 1.0.

- `Parameter & k();`
- `const Parameter & k() const;`

Get the n-parameter. Default value = 1.0.

- `Parameter & n();`
- `const Parameter & n() const;`

2.2.22 class Erf

`#include "QatGenericFunctions/Erf.h"`

Description: Takes the error function of its argument.

Implementation: Calls `std::erf`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Erf();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.23 class Exp

`#include "QatGenericFunctions/Exp.h"`

Description: Takes the exponential function of its argument.

Implementation: Calls `std::exp`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Exp();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.24 class F1D

`#include "QatGenericFunctions/F1D.h"`

Description: `F1D` is an adaptor. Its constructor takes an ordinary function pointer and turns it into an `AbsFunction`, endowed with the normal algebraic and analytic operations of all `AbsFunctions`. The derivative of an `F1D` is taken numerically.

Implementation: Native implementation

Dimensionality: Function of one variable.

Type definitions:

- `typedef double (*FcnPtr)(double);`

Methods: Constructor. Usage example: `F1D f=std::sin`

- `F1D(FcnPtr fcn);`

2.2.25 class F2D

`#include "QatGenericFunctions/F2D.h"`

Description: `F2D` is an adaptor. Its constructor takes an ordinary function pointer (to a function of two double precision arguments) and turns it into an `AbsFunction`, endowed with the normal algebraic and analytic operations of all `AbsFunctions`. Partial derivatives of an `F2D` is taken numerically.

Implementation: Native implementation

Dimensionality: Function of two variable.

Type definitions:

- `typedef double (*FcnPtr)(double,double);`

Methods: Constructor. Usage example: `F2D f=std::atan2`

- `F2D(FcnPtr fcn);`

2.2.26 class FixedConstant

`#include "QatGenericFunctions/FixedConstant.h"`

Description: FixedConstant describes function whose value is fixed and does not depend upon its argument; one can also describe it as a zeroth-order polynomial.

Implementation: Native

Dimensionality: Function of one variable.

Methods: Constructor. Requires the value of the constant.

- `FixedConstant(double value);`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.27 class Gamma

`#include "QatGenericFunctions/.h"`

Description: Implements the gamma function,

$$\Gamma(x) = \int_0^\infty t^{x-1} e^{-t} dt$$

which obeys the relation

$$\Gamma(n) = (n-1)!$$

Implementation: Calls `std::tgamma`.

Dimensionality: Funcion of one variable

Methods: Constructor:

- `Gamma();`

2.2.28 class HermitePolynomial

`#include "QatGenericFunctions/HermitePolynomial.h"`

Description: Implements the Hermite polynomial, $H_{\rm n}(x)$ of order n. These polynomials are orthogonal and with the standard normalization the orthogonality relation is

$$\int_{-\infty}^{\infty} H_n(x)H_m(x)e^{-x^2}dx = \sqrt{\pi}2^n n!\delta_{nm}$$

Normalized Hermite polynomials are denoted as $\tilde{H}_n(x)$ and are both orthogonal and normal. They can be obtained by specifying `TWIDDLE` normalization to the constructor.

Implementation: Native.

Dimensionality: Function of one variable.

Methods: Constructor taking the order n. Normalization type specifies non-normalized (`STD`) or normalized (`TWIDDLE`) orthogonal polynomials.

- `HermitePolynomial(unsigned int n, NormalizationType type=STD);`

Get the order n:

- `unsigned int n() const;`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.29 class IncompleteGamma

`#include "QatGenericFunctions/IncompleteGamma.h"`

Description: Implements one of the following functions; the choice being specfied in the constructor:

$$\begin{aligned}
\Gamma_{Lower}(x;a) &= \int_0^a t^{x-1}e^{-t}dt \\
\Gamma_{Upper}(x;a) &= \int_a^\infty t^{x-1}e^{-t}dt \\
\Gamma_{P}(x;a) &= \frac{1}{\Gamma(a)}\int_0^a t^{x-1}e^{-t}dt \\
\Gamma_{Q}(x;a) &= \frac{1}{\Gamma(a)}\int_a^\infty t^{x-1}e^{-t}dt
\end{aligned}$$

Implementation: Calls `gsl_sf_gamma_inc_e`, `gsl_sf_gamma_inc_e`, or `gsl_sf_gamma_inc_e` from the gnu scientific library (GSL)

Dimensionality: Function of one variable.

Enums:

- `enum Type {UPPER, LOWER, Q, P}`

Methods: Constructor

- `IncompleteGamma(Type type=UPPER);`

Get the parameter a:

- `Parameter & a();`
- `const Parameter & a() const;`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.30 class InterpolatingFunction

`#include "QatGenericFunctions/InterpolatingFunction.h"`

Description: This class implements multiple interpolation schemes, including the Lagrange interpolating polynomial (default), straight line interpolation between points, natural cubic spline interpolation, Non-rounded natural akima spline, cubic spline with periodic boundary conditions, and akima spline with periodic boundary conditions. The choice is made in the constructor. See also: class `InterpolatingPolynomial`, class `CubicSplinePolynomial`.

Implementation: Calls `gsl_spline_init` and `gsl_spline_eval` from the gnu scientific library(GSL).

Dimensionality: Function of one variable.

Enums:

- enum InterpolationType {POLYNOMIAL, CUBIC_SPLINE, CUBIC_SPLINE_PERIODIC, AKIMA_SPLINE, AKIMA_SPLI
  LINEAR};

Methods: Constructor

- `InterpolatingFunction(InterpolationType=POLYNOMIAL);`

Add a point through which to interpolate:

- `void addPoint(double x, double y);`

Get the range:

- `void getRange(double & min, double & max) const;`

Get extreme points maximum and minimum. Abscissa and ordinate:

- `void getExtreme(double & xmin, double & xmax, double &ymin, double &ymax) const;`

2.2.31 class InterpolatingPolynomial

`#include "QatGenericFunctions/InterpolatingPolynomial.h"`

Description: Interpolates between points (ordinate & abscissa) using the Lagrange interpolating polynomial.

Implementation: Native

Dimensionality: Function of one variable.

Methods: Constructor

- `InterpolatingPolynomial();`

Add a new point to the set:

- `void addPoint(double x, double y);`

Get the range:

- `void getRange(double & min, double & max) const;`

Get an estimate of the interpolation error. A positive number:

- `double getDelta() const;`

2.2.32 class LegendrePolynomial

`#include "QatGenericFunctions/LegendrePolynomial.h"`

Description: mplements the Legendre polynomial, $P_{\rm n}(x)$ of order n. These polynomials are orthogonal and with the standard normalization the orthogonality relation is

$$\int_{-1}^{1} P_n(x)P_m(x)dx = \frac{2}{2n+1}\delta_{nm}$$

Normalized Legendre polynomials are denoted as $\tilde{P}_n(x)$ and are both orthogonal and normal. They can be obtained by specifying `TWIDDLE` normalization to the constructor.

Implementation: Calls `gsl_sf_legendre_Pl_e` from the gnu scientific library.

Dimensionality: Function of one variable.

Methods: Constructor. l is the order of the polynomial, normalization type may be `STD` or `TWIDDLE`.

- `LegendrePolynomial(unsigned int l, NormalizationType type=STD);`

Get the order l of the polynomial:

- `unsigned int l() const;`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.33 class LGamma

`#include "QatGenericFunctions/LGamma.h"`

Description: Computes the log of the gamma function (see above).

Implementation: Calls `std::lgamma`.

Dimensionality: Function of one variable.

Methods: Constructor

- `LGamma();`

2.2.34 class Log

`#include "QatGenericFunctions/Log.h"`

Description: Computes the natural (!) log of its argument.

Implementation: Calls `std::log`.

Dimensionality: Function of one variable.

Methods: Constructor

- `Log();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.35 class Mod

Description: Computes the remainder under floored division by a divisor which is specified in the constructor.

Implementation: Native; calls the floor function from the standard math library.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Mod(double y);`

2.2.36 class NormalDistribution

`#include "QatGenericFunctions/.h"`

Description: Implements a normal distribution, or Gaussian distribution. Parameterized by mean and sigma.

Implementation: Native

Dimensionality: Function of one variable.

Methods: Constructor

- `NormalDistribution;`

Get the mean of the normal distribution:

- `Parameter & mean();`
- `const Parameter & mean() const;`

Get the sigma of the NormalDistribution

- `Parameter & sigma();`
- `const Parameter & sigma() const;`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.37 class Pi

`#include "QatGenericFunctions/Pi.h"`

Description: Computes the product of a set of functions:

$$f(x) = \prod_{i=0}^{n-1} g_i(x, ...)$$

Implementation: Native.

Dimensionality: Function of one or more variables, depending upon the functions which compose the product. Note that this is not a direct product, all of the functions $g_i(x, ...)$must have the same dimensionality. Direct products can be implemented with the % operator.

Methods: Constructor

- `Pi;`

Add a function to the product; the following two forms are completely equivalent:

- `void accumulate (const AbsFunction & fcn);`
- `void accumulate (GENFUNCTION fcn);`

Computes the derivative analytically. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.38 class Power

`#include "QatGenericFunctions/Power.h"`

Description: Takes its argument to the n^{th} power, i.e. computes x^n. The exponent n may be double or integral.

Implementation: Calls `std::pow.`

Dimensionality: Function of one variable.

Methods: Constructors:

- `Power(double n);`
- `Power(unsigned int n);`
- `Power(int n);`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.39 class Polynomial

`#include "QatGenericFunctions/Polynomial.h"`

Description: Implements a polynomial function $a_n x^n + a_{n-1} x^{n-1} ... + a_1 x + a_0$. The polynomial is specified by its coefficients which are given in the constructor. The coefficient a_n is given first in the list, followed by $a_{n-1}$and so on down to a_0, which should appear last in the list used to initialize the polynomial.

Implementation: Native.

Dimensionality: Function of one variable.

Methods: Constructors:

- `template <typename ConstIterator> Polynomial(ConstIterator begin, ConstIterator end);`
- `Polynomial(std::initializer_list<double> values);`

Get the roots of this polynomial:

- `std::vector<std::complex<double>> getRoots() const;`

Get the companion matrix (see Press et al, *Numerical Recipes*, 3rd Edition, Cambridge University Press , 2007, section 9.5.4):

- `std::vector<std::complex<double>> getRoots() const;`

Get the coefficients of the derivative of the polynomial:

- `std::vector<double> coefficientsOfDerivative() const;`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.40 class Sgn

`#include "QatGenericFunctions/Sgn.h"`

Description: A function that returns the sign of its argument: -1 for $x<0$; +1 for $x>0$.

Implementation: Native.

Dimensionality: Function of one variable.

Methods: Constructor

- `Sgn;`

2.2.41 class Sigma

`#include "QatGenericFunctions/Sigma.h"`

Description: Computes the sum of a set of functions:

$$f(x) = \sum_{i=0}^{n-1} g_i(x, ...)$$

Implementation: Native.

Dimensionality: Function of one or more variables, depending upon the functions which compose the sum.

Methods: Constructor

- `Sigma;`

Add a function to the sum; the following two forms are completely equivalent:

- `void accumulate (const AbsFunction & fcn);`
- `void accumulate (GENFUNCTION fcn);`

Computes the derivative analytically. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.42 class Sin

`#include "QatGenericFunctions/Sin.h"`

Description: Takes the sine of its argument.

Implementation: Calls `std::sin`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Sin();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.43 class Sinh

`#include "QatGenericFunctions/Sinh.h"`

Description: Takes the hyperbolic sine of its argument.

Implementation: Calls `std::sinh`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Sinh();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.44 class Sqrt

`#include "QatGenericFunctions/Sqrt.h"`

Description: Takes the positive square root of its argument.

Implementation: Calls `std::sqrt`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Sqrt();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.45 class Square

`#include "QatGenericFunctions/Square.h"`

Description: Takes the square of its argument.

Implementation: Native.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Square();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.46 class Tan

`#include "QatGenericFunctions/Tan.h"`

Description: Takes the tangent of its argument.

Implementation: Calls `std::tan`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Tan();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.47 class Tanh

`#include "QatGenericFunctions/Tanh.h"`

Description: Takes the hyperbolic tangent of its argument.

Implementation: Calls `std::tanh`.

Dimensionality: Function of one variable.

Methods: Constructor:

- `Tanh();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const`.

- `Derivative partial (unsigned int) const;`

2.2.48 class TchebyshevPolynomial

`#include "QatGenericFunctions/TchebyshevPolynomial.h"`

Description: Implements the Tchebyshev polynomials of the first kind, $T_{\rm n}(x)$ of order n, and Tchebyshev polynomials of the second kind, $U_{\rm n}(x)$. The choice is made in the constructor. These polynomials are orthogonal and with the standard normalization the orthogonality relation is

$$\begin{aligned}\int_{-1}^{1} T_n(x)T_m(x)\left(1-x^2\right)^{-1/2}dx &= \delta_{nm}\times\left\{\begin{array}{cc}\frac{\pi}{2} & n\delta_{nm}\times\left\{\begin{array}{cc}\frac{\pi}{2} & n\neq 0\\ \pi & n=0\end{array}\right\}\neq 0\\ & \pi \quad n=0\end{array}\right\}\\ \int_{-1}^{1} U_n(x)U_m(x)\left(1-x^2\right)^{1/2}dx &= \frac{\pi}{2}\delta_{nm}\end{aligned}$$

Normalized Tchebyshev polynomials are denoted as $\tilde{T}_n(x)$ (first kind) and $\tilde{U}_n(x)$ (second kind). and are both orthogonal and normal. They can be obtained by specifying `TWIDDLE` normalization to the constructor.

Implementation: Native.

Dimensionality: Function of one variable.

Enums:

- `enum Type {FirstKind, SecondKind};`

Methods: Constructor. The first argument is the order of the polynomial, second argument is the type, third argument is the normalization.

- `TchebyshevPolynomial(unsigned int l, Type type=FirstKind, NormalizationType normType=STD);`

Get the integer variable n:

- `unsigned int n() const;`

Get the type

- `Type type () const;`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.49 class Theta

`#include "QatGenericFunctions/Theta.h"`

Description: Implements the theta function, also known as the Heaviside step function, which is 0 for $x<0$, 1 for $x \geq 0$. This function may be useful in constructing functions whose form in two regions is different. For example, if $f(x) = g(x)$ when $x < a$, and $f(x) = h(x)$ when $x \geq a$, we can construct $f(x)$ as

$$f(x) = \Theta(x-a)g(x) + \Theta(a-x)h(x)$$

Implementation: Native.

Dimensionality: Function of one variable.

Methods: Constructor

- `Theta();`

Computes the derivative analytically. Requires an argument of 0. Called by the method `AbsFunction::prime() const` .

- `Derivative partial (unsigned int) const;`

2.2.50 class Variable

`#include "QatGenericFunctions/Variable.h"`

Description: Variable represents a variable in an expression, or one variable within a set of variables used to form an expression. In the first case, it simply returns its argument. In the second, it returns one of a set of n arguments which are passed in an `Argument` to a function. The dimensionality is determined by the constructor; the constructor with default values of both arguments, builds a simple variable used most frequently to construct functions of one dimension. Below we give examples of both use cases.

```
{
  Variable X;
  GENFUNCTION F=1-X*X;
}
{
  Variable X;
  Sin sin;
  GENFUNCTION F=sin(3*X);
}
// etcetera
```

```
{
    Variable X(0,2),Y(1,2);
    Sin sin;
    Cos cos;
    GENFUNCTION F=sin(X)*cos(Y); // function of two variables.
    GENFUNCTION partialX=F.partial(X);    // returns dF/dX =  cos(X)cos(Y)
    GENFUNCTION partialY=F.partial(Y);      // returns dF/dY = -sin(X)sin(Y)
}
```

The function F in the above example can alternately be constructed with the direct product operation, i.e.

```
{
   Sin sin;
   Cos cos;
   GENFUNCTION F=sin%cos;
}
```

Implementation: Native.

Dimensionality: Function of one or more variables, depending upon the constructor call.

Methods: Constructors. First variable gives the index of the variable within a set of variables, second variable gives the number of variables in the set.

- `Variable(unsigned int selectionIndex=0, unsigned int dimensionality=1);`

Returns the selectionIndex:

- `unsigned int index() const;`

Computes the derivative analytically. Called by the method `AbsFunction::prime() const` (in case of functions of one variable).

- `Derivative partial (unsigned int) const;`

2.2.51 class VoigtDistribution

`#include "QatGenericFunctions/Voigt.h"`

Description: The Voigt distribution is the convolution of a Gaussian with a Lorentzian (or Breit-Wigner, or Cauchy) distribution. It is characterized by the natural width of the Lorentzian and the width of the convolving Gaussian.

Implementation: Native.

Dimensionality: Function of one variable.

Methods: Constructor

- `VoigtDistribution();`

Get the parameter delta (width of Cauchy/Lorentian/Breit-Wigner)

- `Parameter & delta();`

Get the parameter (width of Gaussian/normal)

- `Parameter & sigma();`

2.2.52 Build your own custom function object

The QatGenericFunctions classes are designed to be used by composition. For example, while the function $log_{10}(x)$ does not exist in the library, it can be easily manfactured by using the relationship

$$\log_{10}(x) = \frac{\ln(x)}{\ln(10)};$$

in code,

```
Genfun::Ln ln;
Genfun::GENFUNCTION & log=ln/ln(10.0);
```

In some cases, however, it may be necessary or desirable to create your own function-object. This can be done by inheriting from the class `Genfun::AbsFunction`. The usual steps are necessary: declare you class, inherit publically from the base class, implement all of the required pure virtual functions:

- `virtual double operator() (double argument) const=0;` If your function is one-dimensional, compute and return function value; and if not, throw an exception (suggest std::range_error).
- `virtual double operator() (const Argument &argument) const=0;` Compute and return the function value in case of either one- or multi-dimensional function.

and override any virtual function that you wish to override. The candidates are these:

- `virtual unsigned int dimensionality() const;` You do not need to implement this if your function takes just one variable. Otherwise, override, returning the dimensionality of your function.
- `virtual AbsFunction * clone() const=0;` See below.
- `virtual bool hasAnalyticDerivative() const;` By default, an numerical derivative is computed for all functions. If you wish to return a analytic derivative as a function object, override this function to return `true`.
- `virtual Derivative partial(unsigned int) const;` Override this routine to return an analytic derivative if you so wish.

Two macros are provided for convenience:

- `FUNCTION_OBJECT_DEF(classname)` is used within the class declaration, usually in a header (`.h`) file. It declares the required `virtual AbsFunction * clone() const` method, with the required covariant return type.

- `FUNCTION_OBJECT_IMP(classname)` is used within the class definition, usually in an implementation (`.cpp`) file. It defines the `virtual AbsFunction * clone() const method.`
- `FUNCTION_OBJECT_IMP_INLINE(classname)` is the same as above, but is used if the implmentation is as an inline function, which usually goes in an `.icc` or `.inl` file.

Finally, a command-line tool called `qat-function` is provided for your convenience. It generates all the boilerplate for a new customized function, putting the declaration in a header file and the implementation in a source (`.cpp`) file. The command is set up to generate the boilerplate for a function of one dimension that does not provide an analytic derivative. You may change this as needed. The syntax for the command is:

qat−function FunctionName

The source code and header files `FunctionName.cpp` and `FunctionName.h` appear in the working directory. Edit those files to complete the implementation of your new class, which will then interoperate with all of the other function classes in the `Genfun` namespace.

2.3 QatGenericFunctions: Classes for Numerical Quadrature

This section documents the class `SimpleIntegrator`, which operates together with the class `QuadratureRule` and its subclasses; also the class `GaussIntegrator` which operates together with the class `GaussQuadratureRule` and its subclasses, and also the class `RombergIntegrator`.

The `SimpleIntegrator` applies classical quadrature rules and gives programmers full control over the approximate integration. The `GaussIntegrator` applies Gauss-Legendre, Gauss-Tchebyshev, Gauss-Hermite, or Gauss-Laguerre integration. The `RombergIntegrator` applies Romberg integration, which is an automatic integration scheme which subdivides the mesh iteratively until a convergence criterion is satisfied.

Class tree diagrams appear in Fig. 1 for the `QuadratureRule` class and subclasses and in Fig. 2 for the `GaussQuadratureRule` class and subclasses.

2.3.1 class SimpleIntegrator

```
#include "QatGenericFunctions/SimpleIntegrator.h"
```

Inherits `AbsFunctional`

Description: This class uses a quadrature rule to compute the integral of a function over a specified range, by applying the quadrature rule within a specified number of intervals. If no quadrature rule is provided, then by default this integrator will use the trapezoid rule. If the number of intervals is not specified, the integrator will use a default of 2^6=64.

Methods: Constructor. Constructs an integrator with lower limit `a` and upper limit `b`; applies the quadrature rule `rule` with `nIntervals` intervals.

- ```
  SimpleIntegrator(double a, double b,
  const QuadratureRule &rule =TrapezoidRule(),
  unsigned int nIntervals=64);
  ```

The function call operator integrates the function:

- `virtual double operator () (GENFUNCTION function) const;`

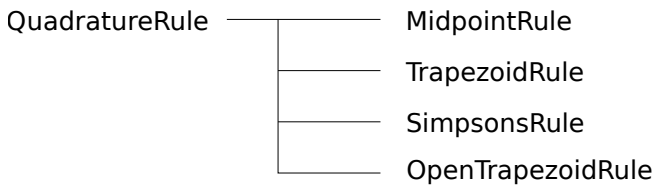


Figure 1: Class Tree Diagram for `QuadratureRule`.
  ```

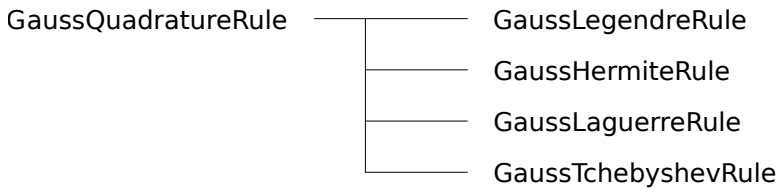


Figure 2: Class Tree Diagram for `GaussQuadratureRule`.

2.3.2 class RombergIntegrator

`#include "QatGenericFunctions/RombergIntegrator.h"`

Inherits `AbsFunctional`

Description: This class uses Romberg integration to compute the integral of a function over a specified range. Either an open or a closed quadrature rule can be used; the closed quadrature rule generally converges faster but the open quadrature rule can be applied in cases where the integrand is infinite at one of the endpoints. Convergence is acheived when either the estimated error absolute value of the integral or the fractional value of the integral are less than the requested precision (10^{-6} by default). For integrals with a value near to zero, a convergence criterion of `ABSOLUTE` is recommended.

Enums:

- `enum Type {OPEN,CLOSED};`
- enum `ConvergenceCriterion {ABSOLUTE,RELATIVE};`

Methods:- Constructor. Constructs an integrator with lower limit `a` and upper limit `b`, with type `OPEN` or `CLOSED`, and convergence criterion `ABSOLUTE` or `RELATIVE`

- `RombergIntegrator(double a, double b, Type type=CLOSED, ConvergenceCriterion criterion=RELATIVE);`

The function call operator integrates the function:

- `virtual double operator () (GENFUNCTION function) const;`

Retrieves the number of function calls to the integrand function, after the integration has been performed:

- `unsigned int numFunctionCalls() const;`

Sets the desired precision. Default is 10^{-6}:

- `void setEpsilon(double eps);`

Sets the maximum number of iterations before bailout. Default is 20 (closed integration) or 14 (open integration)

- `void setMaxIter(unsigned int maxIter);`

Sets the minimum order of integration. Default is 10:

- `void setMinOrder(unsigned int minOrder);`

2.3.3 class QuadratureRule

`#include "QatGenericFunctions/QuadratureRule.h"`

Description: Abstract base class representing a classical quadrature rule. `QuadratureRule`s provide abscissas and weights for integration. They imply extended quadrature rules when used with the `SimpleIntegrator`.

Methods: Constructor:

- `QuadratureRule();`

Access the size of the quadrature rule, i.e. the number of abscissas and weights.

- `unsigned int size() const;`

Access the abscissas:

- `const double & abscissa(unsigned int i) const;`

Access the weights:

- `const double & weight(unsigned int i) const;`

2.3.4 class TrapezoidRule

`#include "QatGenericFunctions/QuadratureRule.h"`

Inherits: `QuadratureRule`

Description: Implements the trapezoid rule, a closed second-order Newton-Cotes quadrature rule.

Methods: Constructor:

- `TrapezoidRule();`

2.3.5 class OpenTrapezoidRule

`#include "QatGenericFunctions/QuadratureRule.h"`

Inherits: `QuadratureRule`

Description: Implements the open trapezoid rule, an open second-order Newton-Cotes quadrature rule.

Methods: Constructor:

- `OpenTrapezoidRule();`

2.3.6 class MidpointRule

`#include "QatGenericFunctions/QuadratureRule.h"`

Inherits: `QuadratureRule`

Description: Implements the midpoint rule, an open second-order Newton-Cotes quadrature rule.

Methods: Constructor:

- `MidpointRule();`

2.3.7 class SimpsonsRule

`#include "QatGenericFunctions/QuadratureRule.h"`

Inherits: `QuadratureRule`

Description: Implements Simpson's rule, a closed fourth-order Newton-Cotes quadrature rule.

Methods: Constructor:

- `SimpsonsRule();`

2.3.8 class GaussIntegrator

`#include "QatGenericFunctions/GaussIntegrator.h"`

Inherits: `AbsFunctional`.

Description: Performs Gaussian integration. The interval over which integration is performed depends upon the integration scheme, which in turn is determined by the `GaussQuadratureRule`. The `GaussLegendreRule` performs Gauss-Legendre integration on the interval [-1,1], the `GaussHermiteRule` performs Gauss-Hermite integration on the interval $[-\infty, \infty]$,etc.

Enums:

- `enum Type {INTEGRATE_DX, INTEGRATE_WX_DX}`

Methods: Constructor. The GaussQuadratureRule determines the integration scheme and upper and lower bounds of the integral. The type determines whether the function $f(x)$ is to be integrated on its own, $\int f(x)dx$, or whether it is to be integrated together with the weight function $w(x)$, i.e. integrated $\int f(x)w(x)dx$. A type of `INTEGRATE_DX` specfies the former case, `INTEGRATE_WX_DC` the latter case.

- `GaussIntegrator(const GaussQuadratureRule & rule, Type=INTEGRATE_DX);`

The function call operator integrates the function:

- `virtual double operator () (GENFUNCTION function) const;`

2.3.9 class GaussQuadratureRule

`#include "QatGenericFunction/GaussQuadratureRule.h"`

Description: A abstract base class for Gaussian integration. This class holds much of the functionality for determining the abscissas and weights for Gaussian integration, so that subclasses need only furnish few pieces of information concerning the orthogonal polynomials upon which their rule is based. These include the coefficients a_i,b_i,c_i of the three-term recurrence relation; the natural logarithm of the normalization constant, the limits of integration, the weight function, and the integral of the weight function. The GaussIntegrator uses the weight function, the limits, the abscissas, and the weights in carrying out the integration.

Methods: Constructor. Note that since the class is abstract, only the subclasses can construct.

- `GaussQuadratureRule(const std::string & name, unsigned int npoints,`
  `double min, double max, GENFUNCTION W);`

Access the name:

- `const std::string & name() const;`

Access limits of integration associated with the integration scheme:

- `double min() const;`
- `double max() const;`

Coefficients in the three-term recurrence relation $p_{i+1}(x) = (a_i x + b_i)\, p_i(x) - c_i p_{i-1}(x)$; provided by subclasses.

- `virtual double a(unsigned int i) const=0;`
- `virtual double b(unsigned int i) const=0;`
- `virtual double c(unsigned int i) const=0;`

Access the natural log of the normalization constant $\log(N_i) = \log(\int p_i(x)p_i(x)dx$. The logarithm is taken since the constants can become very large for some of the orthogonal polynomials.

- `virtual double lognorm(unsigned int i) const=0;`

Access the integrated weight; provided by subclasses: $\mu = \int w(x)dx$

- `virtual double mu() const=0;`

Access the weight function $w(x)$; provided by subclasses:

- `GENFUNCTION weightFunction() const;`

Access the number of points:

- `unsigned int nPoints() const;`

Access abscissa

- `const double & abscissa(unsigned int i) const;`

Access weight:

- `const double & weight(unsigned int i) const;`

2.3.10 class GaussLegendreRule

`#include "QatGenericFunctions/GaussQuadratureRule.h"`

Inherits: `GaussQuadratureRule`

Description: A Gaussian quadrature rule implementing Gauss-Legendre integration on the interval $[-1, 1]$, based upon Legendre polynomials. The weight function for Legendre polynomials is $w(x) = 1.0$; i.e. it is flat.

Methods: Constructor. Argument to the constructor is the number of points.

- `GaussLegendreRule(unsigned int npoints);`

Coefficients in the three-term recurrence relation $p_{i+1}(x) = (a_i x + b_i)\, p_i(x) - c_i p_{i-1}(x)$;provided by subclasses. For Legendre polynomials $a_i = \frac{2i+1}{i+1}$; $b_i = 0$, and $c_i = \frac{i}{i+1}$

- `virtual double a(unsigned int i) const;`
- `virtual double b(unsigned int i) const;`
- `virtual double c(unsigned int i) const;`

Access the natural log of the normalization constant $\log(N_i) = \log(\int p_i(x)p_i(x)dx$. For Legendre polynomials $\log N_i = \frac{1}{2}\log\frac{2}{2i+1}$.

- `virtual double lognorm(unsigned int i) const=0;`

Access the integrated weight: $\mu = \int w(x)dx$. For Legendre polynomials $\mu = 2$

- `virtual double mu() const;`

2.3.11 class GaussHermiteRule

`#include "QatGenericFunctions/GaussQuadratureRule.h"`

Inherits: `GaussQuadratureRule`

Description: A Gaussian quadrature rule implementing Gauss-Hermite integration on the interval $[-\infty, \infty]$, based upon Hermite polynomials. The weight function for Hermite polynomials is $w(x) = e^{-x^2}$.

Methods: Constructor. Argument to the constructor is the number of points.

- `GaussHermiteRule(unsigned int npoints);`

Coefficients in the three-term recurrence relation $p_{i+1}(x) = (a_i x + b_i)\, p_i(x) - c_i p_{i-1}(x)$;provided by subclasses. For Hermite polynomials $a_i = 2$; $b_i = 0$, and $c_i = 2i$

- `virtual double a(unsigned int i) const;`
- `virtual double b(unsigned int i) const;`
- `virtual double c(unsigned int i) const;`

Access the natural log of the normalization constant $\log(N_i) = \log(\int p_i(x) p_i(x) w(x) dx$. For Hermite polynomials $\log N_i = \frac{1}{2}(\frac{1}{2}\log\pi + i\log 2 + \log(\Gamma(\mathrm{i}+1))$.

- `virtual double lognorm(unsigned int i) const=0;`

Access the integrated weight: $\mu = \int w(x)dx$. For Hermite polynomials $\mu = \sqrt{\pi}$

- `virtual double mu() const;`

2.3.12 class GaussTchebyshevRule

`#include "QatGenericFunctionsGaussQuadratureRule.h"`

Inherits: `GaussQuadratureRule`

Description: A Gaussian quadrature rule implementing Gauss-Tchebyshev integration on the interval $[-1, 1]$, based upon Tchebyshev polynomials of the first or second kind, the choice being taken with the constructor. The weight function for Tchebyshev polynomials of the first kind is $w(x) = \left(1 - x^2\right)^{-\frac{1}{2}}$; for Tchebyshev polynomials of the second kind is $w(x) = \left(1 - x^2\right)^{\frac{1}{2}}$.

Methods: Constructor. Argument to the constructor is the number of points, and the type, which can take values `TchebyshevPolynomial::FirstKind` or `TchebyshevPolynomial::SecondKind`.

- `GaussTchebyshevRule(unsigned int npoints, TchebyshevPolynomial::Type type);`

Coefficients in the three-term recurrence relation $p_{i+1}(x) = (a_i x + b_i)\, p_i(x) - c_i p_{i-1}(x)$;provided by subclasses. For Tchebyshev polynomials of the first kind, $a_0 = 1$;$a_i = 2$ $(i \neq 0)$ $b_i = 0$, and $c_i = 1$; for Tchebyshev polynomials of the second kind, $a_i = 2$, $b_i = 0$, and $c_i = 1$; for

- `virtual double a(unsigned int i) const;`
- `virtual double b(unsigned int i) const;`
- `virtual double c(unsigned int i) const;`

Access the natural log of the normalization constant $\log(N_i) = \log(\int p_i(x) p_i(x) w(x) dx$. For Tchebyshev polynomials of the first kind, $\log N_i = \frac{1}{2}\log\pi$ if $i = 0$ and $\log N_i = \frac{1}{2}\log\frac{\pi}{2}$ if $i \neq 0$; for Tchebyshev polynomials of the second kind, $\log N_i = \frac{1}{2}\log\frac{\pi}{2}$.

- `virtual double lognorm(unsigned int i) const=0;`

Access the integrated weight: $\mu = \int w(x)dx$. For Tchebyshev polynomials of the first kind $\mu = \pi$; and For Tchebyshev polynomials of the second kind $\mu = \frac{\pi}{2}$.

- `virtual double mu() const;`

2.3.13 class GaussLaguerreRule

`#include "QatGenericFunctions/GaussQuadratureRule.h"`

Inherits: `GaussQuadratureRule`

Description: A Gaussian quadrature rule implementing Gauss-Laguerre integration on the interval $[0, \infty]$, based upon Laguerre polynomials. The weight function for Legendre polynomials of degree α is $x^\alpha e^{-x}$.

Methods: Constructor. Argument to the constructor is the number of points and the constant α.

- `GaussLaguerreRule(unsigned int npoints, double alpha);`

Coefficients in the three-term recurrence relation $p_{i+1}(x) = (a_i x + b_i)\, p_i(x) - c_i p_{i-1}(x)$;provided by subclasses. For Laguerre polynomials $a_i = \frac{-1}{i+1}$; $b_i = \frac{2i+\alpha+1}{i+1}$, and $c_i = \frac{i+\alpha}{i+1}$

- `virtual double a(unsigned int i) const;`
- `virtual double b(unsigned int i) const;`
- `virtual double c(unsigned int i) const;`

Access the natural log of the normalization constant $\log(N_i) = \log(\int p_i(x) p_i(x) dx$. For Laguerre polynomials $\log N_i = \frac{1}{2}\left(\log\Gamma(i+\alpha+1) - \log\Gamma(i+1)\right)$.

- `virtual double lognorm(unsigned int i) const=0;`

Access the integrated weight: $\mu = \int w(x)dx$. For Laguerre polynomials $\mu = \Gamma(\alpha + 1)$

- `virtual double mu() const;`

2.4 QatGenericFunctions: Classes for the numerical solution of ordinary differential equations

In this section we describe classes for the solution of systems of ordinary, first-order, autonomous ordinary differential equations (ODEs) . These have the form:

$$\begin{aligned} \frac{dy_1}{dx} &= f_1(y_1,y_2,...y_n) \\ \frac{dy_2}{dx} &= f_2(y_1,y_2,...y_n) \\ ... &= ... \\ \frac{dy_n}{dx} &= f_3(y_1,y_2,...y_n) \end{aligned}$$

which can be written more compactly as

$$\frac{d\vec{y}}{dx} = \vec{f}(\vec{y})$$

Note that any higher order ODE can be reduced to a set of first-order ODESs, and also that nonautonomous set of first-order ODEs can be reduced to an autonomous set, see for example John Butcher, *Numerical Methods for Ordinary Differential Equations*, John Wiley & sons, West Sussex England (2008). These functions, together with the initial values of the solution vector $\vec{y}_0$, determine the solution $\vec{y}(x)$. The classes available for flexibly finding an approximate solution therefore *create functions*. The main class one is the `RKIntegrator`, which internally uses one or more *steppers*. Users can instantiate their own steppers in order to control the stepping algorithm or modify tolerances and other algorithmic parameters. The initial values $\vec{y}_0$ are parameters of the solutions. Other parameters, entering via the differential equations, maybe be involved as well; if these are required they should be created through a call to `RKIntegrator::createControlParameter` so that they may be managed by the `RKIntegrator` and watched for modification. The approximate solutions are cached in memory for the sake of speed, and any change to starting value parameters or managed control parameters changes the solution–and therefore also invalidates the cache. A class tree diagram is shown in Fig. 3. Two kinds of steppers are provide, a `SimpleRKStepper` implementing fixed stepsize and an `AdaptiveRKStepper` which adjustes the stepsize for an approximately equal tolerance at each step. The latter functions interoperates with either the `StepDoublingRKStepper` or the `EmbeddedRKStepper`, both of which can provide error estimates at each step. The actual stepping algorithm is set by one of the `ButcherTableau` classes (for either the `SimpleRKStepper` or the `StepDoublingRKStepper`) or an `ExtendedButcherTableau` (for the `EmbeddedRKStepper`).

2.4.1 class RKIntegrator

`#include "QatGenericFunctions/RKIntegrator.h"`

Description: The `RKIntegrator` is the main class for ODE integration. To use it, you add differential equations (GENFUNCTIONS of N variables), one equation for each variable, specifying at the same time a starting value and range, in case the starting value should be an adjustable parameter of the solution. When you are finished adding the required N differential equations, you may request the "solution", which is an `const AbsFunction *`. The `RKIntegrator` can also manufacture `Parameters`; these can be used like ordinary `Parameters` to construct the differential equations, except that the caching mechanism is informed when the values of these managed `Parameters` change. It is important to note that the solutions are destroyed together with the `RKIntegrator`, which must therefore continue to exist for as long as the solutions are in use.

Methods: Constructor. If no `RKIntegrator::RKStepper` is specified, the `RKIntegrator` creates an `AdaptiveRKStepper` with default values of all algorithmic parameters. This in turn uses the `EmbeddedRKSTepper` with a `CashKarpXtTableau`.

- `RKIntegrator(const RKIntegrator::RKStepper *stepper=NULL);`

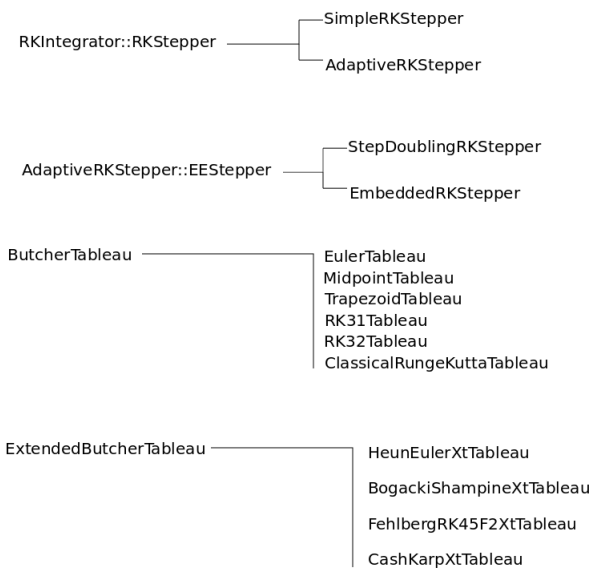


Figure 3: Class tree diagram for ordinary differential equation solving.

Add a differential equation governing the evolution of the next variable with independent variable x. This function returns a `Parameter` representing the starting value of that variable. The `Parameter` may be held constant, modified; any modification of the `Parameter` value wipes out the cache since the approximate solution changes. The `Parameter` is destroyed with the `RKIntegrator`.

- ```
  Parameter * addDiffEquation (const AbsFunction * diffEquation,
  const std::string & variableName="anon",
  double defStartingValue=0.0,
  double startingValueMin=0.0,
  double startingValueMax=0.0);
  ```

Create a control parameter. The `Parameter` is checked for modification when the function is evaluated and if it has changed the cache is wiped and the approximate solution reevaluated. If you are considering parameterizing the ODE solutions via their defining equations, you should use this method to create the `Parameter`s. The `Parameter` is destroyed with the `RKIntegrator`.

- ```
  Parameter *createControlParameter (const std::string & variableName="anon",
  double defStartingValue=0.0, double startingValueMin=0.0, double startingValueMax=0.0);
  ```

Retrieve the solution to the ODE. This function will now actually change as parameters are changed; this includes both control parameters and starting value parameters. The function is destroyed with the `RKIntegrator`.

- ```
  const RKFunction *getFunction(unsigned int i) const;
  ```
- ```
  const RKFunction *getFunction(const Variable & v) const;
  ```

2.4.2 class SimpleRKStepper

`#include "QatGenericFunctions/SimpleRKStepper.h"`

Inherits: `RKIntegrator::RKStepper`

Description: The `SimpleRKStepper` implements a fixed stepsize together with an integration method specified by a ButcherTableau.

Methods: Constructor, specifying the `ButcherTableau` and the stepsize:

- `SimpleRKStepper(const ButcherTableau & tableau, double stepsize);`

2.4.3 class AdaptiveRKStepper

#include `"QatGenericFunctions/AdaptiveRKStepper.h"`

Inherits: `RKIntegrator::RKStepper`

Description:

Methods: Constructor. If no `AdaptiveRKStepper::EEStepper` is specified, it will create its own `EmbeddedRKStepper` using a `CashKarpXtTableau`.

- `AdaptiveRKStepper(const AdpativeRKStepper::EEStepper *eeStepper=NULL);`

The tolerance (default 10^{-6})

- `double & tolerance();`
- `const double & tolerance() const;`

The starting stepsize (default 0.01):

- `double & startingStepsize();`
- `const double & startingStepsize() const;`

The safety factor. Step size increases are moderated by this factor (default 0.9):

- `double & safetyFactor();`
- `const double & safetyFactor() const;`

The minimum factor by which a step size is decreased (default 0.0):

- `double & rmin();`
- `const double & rmin() const;`

The maximum factor by which a step size is increased (default 5.0):

- `double & rmax();`
- `const double & rmax() const;`

2.4.4 class StepDoublingRKStepper

`#include "QatGenericFunctions/StepDoublingRKStepper.h"`

Inherits: `AdaptiveRKStepper::EEStepper`

Description: An error-estimating stepping algorithm that works by taking one half-step and one full step. Error extracted from the two results.

Methods: Constructor:

- `StepDoublingRKStepper(const ButcherTableau & tableau);`

2.4.5 class EmbeddedRKStepper

`#include "QatGenericFunctions/StepDoublingRKStepper.h"`

Inherits: `AdaptiveRKStepper::EEStepper`

Description: An error-estimating stepping algorithm that works by computing two results from intermediate evaluations at each step, each result having its own order-of-convergence. The algorithm is steered by a `ExtendedButcherTableau`.

Methods: Constructor, taking an `ExtendedButcherTableau`.

- `EmbeddedRKStepper(const ExtendedButcherTableau & tableau=CashKarpXtTableau() );`

2.4.6 class ButcherTableau

`#include "QatGenericFunctions/ButcherTableau.h"`

Inherits: `GaussQuadratureRule`

Description: In numerical analysis, a Butcher tableau is table of numbers in a particular format which completely specifies a Runge-Kutte integration scheme. Butcher Tableau are described in John Butcher, *Numerical Methods for Ordinary Differential Equations*, John Wiley & sons, West Sussex England (2008). The general form is :

$$\begin{pmatrix} \mathbf{c} & \boldsymbol{A} \\ & \mathbf{b}^T \end{pmatrix}$$

where $\mathbf{A}$ is a matrix and $\mathbf{b}$, $\mathbf{c}$ are column vectors. The `ButcherTableau` class presents itself as an empty structure that the user has to fill up. One can blithely fill write into any element of $\mathbf{A}$, b, or $\mathbf{c}$. Space is automatically allocated. The class is not normally used directly; one rather uses subclasses such as `EulerTableau`, `MidpointTableau`... (see Fig. 3) which initialize the internals in their constructor; however if one wishes to customize the integration e.g. with higher order methods he/she can do so by subclassing `ButcherTableau.`

Note: in the current implementation, the matrix $\mathbf{A}$ must be lower-diagonal, precluding tableaux which implement implicit integration schemes.

Methods: Constructor:

- `ButcherTableau(const std::string &name, unsigned int order);`

Returns the name:

- `const std::string & name() const;`

Returns the order of integration:

- `unsigned int order() const;`

Returns the number of steps in the integration procedure

- `unsigned int nSteps() const;`

Write access to elements:

- `double & A(unsigned int i, unsigned int j);`
- `double & b(unsigned int i);`
- `double & c(unsigned int i);`

Read access to elements:

- `const double & A(unsigned int i, unsigned int j) const;`
- `const double & b(unsigned int i) const;`
- `const double & c(unsigned int i) const;`

2.4.7 class EulerTableau

`#include "QatGenericFunctions/ButcherTableau.h"`

Inherits: `ButcherTableau`

Description: Implements a Butcher Tableau for the Euler method of ode solution, a first order integration method. See John Butcher, *Numerical Methods for Ordinary Differential Equations*, John Wiley & sons, West Sussex England (2008) for numerical values.

Methods: Constructor:

- `EulerTableau`

2.4.8 class MidpointTableau

`#include "QatGenericFunctions/ButcherTableau.h"`

Inherits: `ButcherTableau`

Description: Implements a Butcher tableau for the midpoint (or RK22) method of ode solution, a second order integration method. See John Butcher, *Numerical Methods for Ordinary Differential Equations*, John Wiley & sons, West Sussex England (2008) for numerical values.

Methods: Constructor:

- `MidpointTableau`

2.4.9 class TrapezoidTableau

```
#include "QatGenericFunctions/ButcherTableau.h"
```

Inherits: `ButcherTableau`

Description: Implements a Butcher Tableau for the Trapezoid (or RK21) method of ode solution, a second order integration method. See John Butcher, *Numerical Methods for Ordinary Differential Equations*, John Wiley & sons, West Sussex England (2008) for numerical values.

Methods: Constructor:

- `TrapezoidTableau`

2.4.10 class RK31Tableau

```
#include "QatGenericFunctions/ButcherTableau.h"
```

Inherits: `ButcherTableau`

Description: Implements a Butcher Tableau for the RK31 method of ode solution, a second order integration method . See John Butcher, *Numerical Methods for Ordinary Differential Equations*, John Wiley & sons, West Sussex England (2008) for numerical values.

Methods: Constructor:

- `RK31Tableau`

2.4.11 class RK32Tableau

```
#include "QatGenericFunctions/ButcherTableau.h"
```

Inherits: `ButcherTableau`

Description: Implements a Butcher Tableau for the RK32 method of ode solution, a third order integration method. See John Butcher, *Numerical Methods for Ordinary Differential Equations*, John Wiley & sons, West Sussex England (2008) for numerical values.

Methods: Constructor:

- `RK32Tableau`

2.4.12 class ClassicalRungeKuttaTableau

`#include "QatGenericFunctions/ButcherTableau.h"`

Inherits: `ButcherTableau`

Description: Implements a Butcher Tableau for the Classical fourth-order Runge-Kutta method of ode solution, also called RK41. See John Butcher, *Numerical Methods for Ordinary Differential Equations*, John Wiley & sons, West Sussex England (2008) for numerical values.

Methods: Constructor:

- `ClassicalRungeKuttaTableau`

2.4.13 class ThreeEightsRuleTableau

`#include "QatGenericFunctions/ButcherTableau.h"`

Inherits: `ButcherTableau`

Description: Implements a Butcher Tableau for the three eights rule of ode solution, a fourth-order integration method. See wikipedia for numerical values.

Methods: Constructor:

- `ThreeEightsRuleTableau`

2.4.14 class ExtendedButcherTableau

`#include "QatGenericFunctions/ExtendedButcherTableau.h"`

Description: An extended Butcher tableau is similar to a Butcher tableau, except it is used by an embedded stepping algorithm, in which each step terminates with two estimates of the solution, by combining the intermediate values in two different ways. Typically these alternate estimates are characterized by different order of convergence; the availability of a second estimate of the function allows for error estimation, which is exploited by the `EmbeddedRKStepper`, within which these classes are used. See John Butcher, *Numerical Methods for Ordinary Differential Equations*, John Wiley & sons, West Sussex England (2008). An extended Butcher tableau has the form:

$$\begin{pmatrix} \mathbf{c} & \boldsymbol{A} \\ & \mathbf{b}^T \\ & \hat{\mathbf{b}}^T \end{pmatrix}$$

which is similar to the form of an ordinary Butcher tableau with the transpose of an added column vector, $\hat{\mathbf{b}}$, in the final row. The class is not normally used directly; one rather uses subclasses such as `HeunEulerXtTableau`, `FehlbergRK45F2XtTableau`... (see Fig. 3) which initialize the internals in their constructor; however if one wishes to customize the integration e.g. with higher order methods he/she can do so by subclassing `ExtendedButcherTableau.`

Methods: Constructor:

- `ExtendedButcherTableau(const std::string &name, unsigned int order, unsigned int orderHat);`

Returns the name:

- `const std::string & name() const;`

Returns the order(s) of integration:

- `unsigned int order() const;`
- `unsigned int orderHat() const;`

Returns the number of steps in the integration procedure

- `unsigned int nSteps() const;`

Write access to elements:

- `double & A(unsigned int i, unsigned int j);`
- `double & b(unsigned int i);`
- `double & bHat(unsigned int i);`
- `double & c(unsigned int i);`

Read access to elements:

- `const double & A(unsigned int i, unsigned int j) const;`
- `const double & b(unsigned int i) const;`
- `const double & bHat(unsigned int i) const;`
- `const double & c(unsigned int i) const;`

2.4.15 class HeunEulerXtTableau

`#include "QatGenericFunctions/ExtendedButcherTableau.h"`

Inherits: `ExtendedButcherTableau`

Description: Implements an extended Butcher tableau for the Heun-Euler method, which generates a second-order estimate and a first-order estimate of the solution. See wikipedia for numerical values.

Methods: Constructor:

- HeunEulerXtTabeau()

2.4.16 class BogackiShampineXtTableau

`#include "QatGenericFunctions/ExtendedButcherTableau.h"`

Inherits: `ExtendedButcherTableau`

Description: Implements an extended Butcher tableau for the Bogacki-Shampine method, which generates third-order estimate and a second-order estimate of the solution. See wikipedia for numerical values.

Methods: Constructor:

- BogackiShampineXtTabeau()

2.4.17 class FehlbergRK45F2XtTableau

`#include "QatGenericFunctions/ExtendedButcherTableau.h"`

Inherits: `ExtendedButcherTableau`

Description: Implements an extended Butcher tableau for the FehlbergRK45F2 method, which generates a fourth-order estimate and a fifth-order estimate of the solution. See wikipedia for numerical values.

Methods: Constructor:

- FehlbergRK45F2XtTabeau()

2.4.18 class CashKarpXtTableau

`#include "QatGenericFunctions/ExtendedButcherTableau.h"`

Inherits: `ExtendedButcherTableau`

Description: Implements an extended Butcher tableau for the Cash-Karp method, which generates a fourth-order estimate and a fifth-order estimate of the solution. See wikipedia for numerical values.

Methods: Constructor:

- CashKarpXtTabeau()

2.5 QatGenericFunctions: Classes for the solution of Hamiltonian Systems

The classes in this section apply the automatic derivative system inherent in `QatGenericFunctions`, with the numerical integration methods of Section 2.4, to classical Hamiltonian systems with no explicit dependence on the time. Such systems are governed by a Hamiltonian, which is a function of the phase space variables (generalized coordinates $q_i$and generalized momenta p_i, $i = 1, N$) specific to the particular classical system; for example, for the harmonic oscillator in one dimension one has

$$H = \frac{1}{2}q^2 + \frac{1}{2}p^2$$

The system is solved by first obtaining Hamiltons's equations from the Hamiltonian:

$$\begin{aligned} \frac{dq_i}{dt} &= \frac{\partial H}{\partial p_i} \\ \frac{dp_i}{dt} &= -\frac{\partial H}{\partial q_i} \end{aligned}$$

and then integrating Hamilton's equations, which are first-order ODEs in a standard form. Thus the time evolution of a classical system are fully determined by the Hamiltonian and the starting values of all generalized coordinates and momenta. The classes in this section allow to obtain the full time evolution of a system by expressing the Hamiltonian in `QatGenericFunctions` and specifying the starting values for all the coordinates and momenta. The output is a set of functions, one for each generalized coordinate and one for each generalized momenta. The main class `Classical::RungeKuttaSolver` is a `Classical::Solver`. Its destructor also destroys all of the functions and parameters (the starting values) that it has created. All of the classes in this subsection are scoped within the namespace `Classical`.

2.5.1 class Classical::Solver

`#include "QatGenericFunctions/ClassicalSolver.h"`

Description: Abstract base class for Hamiltonian solvers.

Methods: Constructor:

- `Solver();`

Returns the time evolution for a variable (which should be q_i or p_i) :

- `virtual Genfun::GENFUNCTION equationOf(const Genfun::Variable & v) const;`

Returns the phase space

- `virtual const PhaseSpace & phaseSpace() const;`

Returns the Hamiltonian (function of the 2N phase space variables).

- `virtual Genfun::GENFUNCTION hamiltonian() const;`

Returns the energy (function of time):

- `virtual Genfun::GENFUNCTION energy() const;`

This is in the case that the user needs to edit starting values or parameterize the Hamiltonian.

- `virtual Genfun::Parameter *takeQ0(unsigned int index);`
- `virtual Genfun::Parameter *takeP0(unsigned int index);`
- ```
  virtual Genfun::Parameter *createControlParameter(const std::string & variableName="anon",
  double defStartingValue=0.0,
  double startingValueMin=0.0,
  double startingValueMax=0.0) const;
  ```

2.5.2 class Classical::RungeKuttaSolver

`#include "QatGenericFunctions/RungeKuttaClassicalSolver.h"`

Inherits: `Classical::Solver`

Description: The `Classical::RungeKutta` solver implements the interface `Classical::Solver`, obtaining solutions to Hamiltonian systems using Runge-Kutta integration. Internally it uses `RKIntegrator`.

Methods: Constructor, which takes a Hamiltonian `H`, a properly configured `PhaseSpace`, and (optionally) an `RKIntegrator::RKStepp` which allows users full control over the integration scheme and algorithmic parameters thereof.

- ```
  RungeKuttaSolver(Genfun::GENFUNCTION H,
  const PhaseSpace & phaseSpace,
  const Genfun::RKIntegrator::RKStepper *stepper=NULL);
  ```

2.5.3 class Classical::PhaseSpace

`#include "QatGenericFunctions/PhaseSpace.h"`

Description: `PhaseSpace` is a convenience routine for setting up a set of $2N$ `Genfun::Variable`s, half of which are generalized coordinates, the other half of which are generalized momenta; and also for setting the starting values of these components. The nested class `PhaseSpace::Component` behaves like an array of `Genfun::Variable`s, the i^{th}element of which can be obtained from the subscript operator, `operator []`.

Nested classes: `PhaseSpace::Component`

Methods: Constructor. NDIM gives the number of generalized coordinates, which is also equal to the number of generalized momenta:

- `PhaseSpace(unsigned int DIM);`

Get the dimensionality of the phase space (number of generalized coordinates, which is also equal to the number of generalized momenta):

- `unsigned int dim() const;`

Get the coordinates, which behaves like an array of Genfun::`Variables:`

- `const Component & coordinates() const;`

Get the momenta, which behaves like an array of Genfun::`Variables`:

- `const Component & momenta() const;`

Set starting values for the coordinates or momenta:

- `void start (const Genfun::Variable & variable, double value);`

Get starting values for the coordinates or momenta:

- `double startValue(const Genfun::Variable & component) const ;`

2.6 QatGenericFunctions: Classes for root finding

The classes in this section can be used to search for the roots of a function. If the function you are dealing with is a polynomial, consider using `Genfun::Polynomial` to obtain the roots in a more robust way.

2.6.1 RootFinder

#include "QatGenericFunctions/RootFinder.h"

Description: `RootFinder` is the base class for the classes `Bisection`, `NewtonRaphson`, and `DeflatedNewtonRaphson`, which all attempt to locate the roots of a function.

Methods: Return a root, starting from an estimated value:

- `virtual double root(double estimatedRoot) const=0;`

Read only access to upper and lower bounds:

- `const double & lowerBound() const;`
- `const double & upperBound() const;`

Read-write access to upper and lower bounds:

- `double & lowerBound();`
- `double & upperBound();`

2.6.2 Bisection

#include "QatGenericFunctions/RootFinder.h"

Description: `Bisection` is a class for root finding using the bisection algorithm.

Methods: Constructor, taking the function whose root is to be found, the tolerance with which the answer is desired, and the maximum number of function calls permitted:

- `Bisection(GENFUNCTION P, double EPS=1.0E-12, unsigned int MAXCALLS=100);`

Return a root, starting from an estimated value:

- `virtual double root(double estimatedRoot) const=0;`

Return the number of calls to the function which has been made during the previous root-finding operation.

- `unsigned int nCalls() const;`

Read only access to upper and lower bounds:

- `const double & lowerBound() const;`
- `const double & upperBound() const;`

Read-write access to upper and lower bounds:

- `double & lowerBound();`
- `double & upperBound();`

2.6.3 Newton-Raphson

#include "QatGenericFunctions/RootFinder.h"

Description: `NewtonRaphson` is a class for root finding using the Newton-Raphson algorithm.

Methods: Constructor, taking the function whose root is to be found, the tolerance with which the answer is desired, and the maximum number of function calls permitted:

- `NewtonRaphson(GENFUNCTION P, double EPS=1.0E-12, unsigned int MAXCALLS=100);`

Return a root, starting from an estimated value:

- `virtual double root(double estimatedRoot) const=0;`

Return the number of calls to the function which has been made during the previous root-finding operation.

- `unsigned int nCalls() const;`

Read only access to upper and lower bounds:

- `const double & lowerBound() const;`
- `const double & upperBound() const;`

Read-write access to upper and lower bounds:

- `double & lowerBound();`
- `double & upperBound();`

2.6.4 DeflatedNewtonRaphson

#include "QatGenericFunctions/RootFinder.h"

Description: `DeflatedNewtonRaphson` is a class for root finding using the deflated Newton-Raphson algorithm. The class keeps a copy of the original function, from which it removes each root which has been successfully located during a call to the `root` function. This is known as deflation. Subsequent calls to the `root` function are meant to return a unique root.

Methods: Constructor, taking the function whose root is to be found, the tolerance with which the answer is desired, and the maximum number of function calls permitted:

- `DeflatedNewtonRaphson(GENFUNCTION P, double EPS=1.0E-12, unsigned int MAXCALLS=100);`

Return a root, starting from an estimated value:

- `virtual double root(double estimatedRoot) const=0;`

Return the number of calls to the function which has been made during the previous root-finding operation.

- `unsigned int nCalls() const;`

Read only access to upper and lower bounds:

- `const double & lowerBound() const;`
- `const double & upperBound() const;`

Read-write access to upper and lower bounds:

- `double & lowerBound();`
- `double & upperBound();`

3 QatPlotWidgets Reference Manual

`QatPlotWidgets` is a library for plotting, which is based upon the Qt library , a toolkit for graphical user interfaces and two-dimensional graphics. `QatPlotWidgets` contains a number of customized `QtWidgets`, the main one being `PlotView`, which is an x-y plotter that can handle functions, histograms, and a variety of other plottable objects. The library also contains several dialogs used internally by `PlotView`, which are "internal" classes will not be documented here. Other user classes which may be of use to users are the `RangeDivider` classes, which allow for linear, logarithmic, or custom range division. Class tree diagrams for these classes appear in Fig. 4

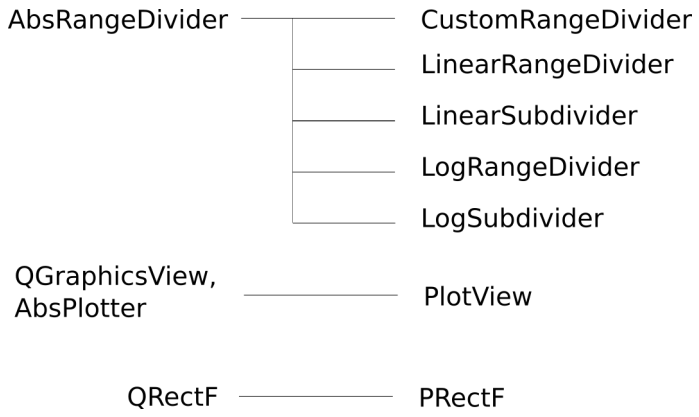


Figure 4: Class Tree Diagram for the main classes of the QatPlotWidget library.

3.0.1 class PlotView

`#include "QatPlotWidgets/PlotView.h"`

Inherits: `QGraphicsView` (`Qt` library), `AbsPlotter` (`QatPlotting`)

Description `PlotView` is the main plotter class. Since it inherits from `QWidget`, it can easily be incorporated into a `Qt` widget hierarchy; since it is a `QGraphicsView`, arbitrary graphics may be layered onto the `PlotView`. Various `Plottables` (See Section 4) can be added to the `PlotView` using the `add(Plottable *)` method. These `Plottables` must remain in scope while the `PlotView` is active. The `Plottables` are drawn within the `PlotView`. And example is shown in Fig. 5. The `PlotView` has many facilities for changing the style and labelling of plots. Many of these can accessed interactively, too. Right-clicking on the `PlotView` opens a dialog enabling to change the range, the type of axes (linear or logarithmic), and the domain and range of the plotter.

Enums:

- `enum Style {LINEAR,LOG};`

Methods: Constructor:

- `PlotView(QWidget *parent=0);`

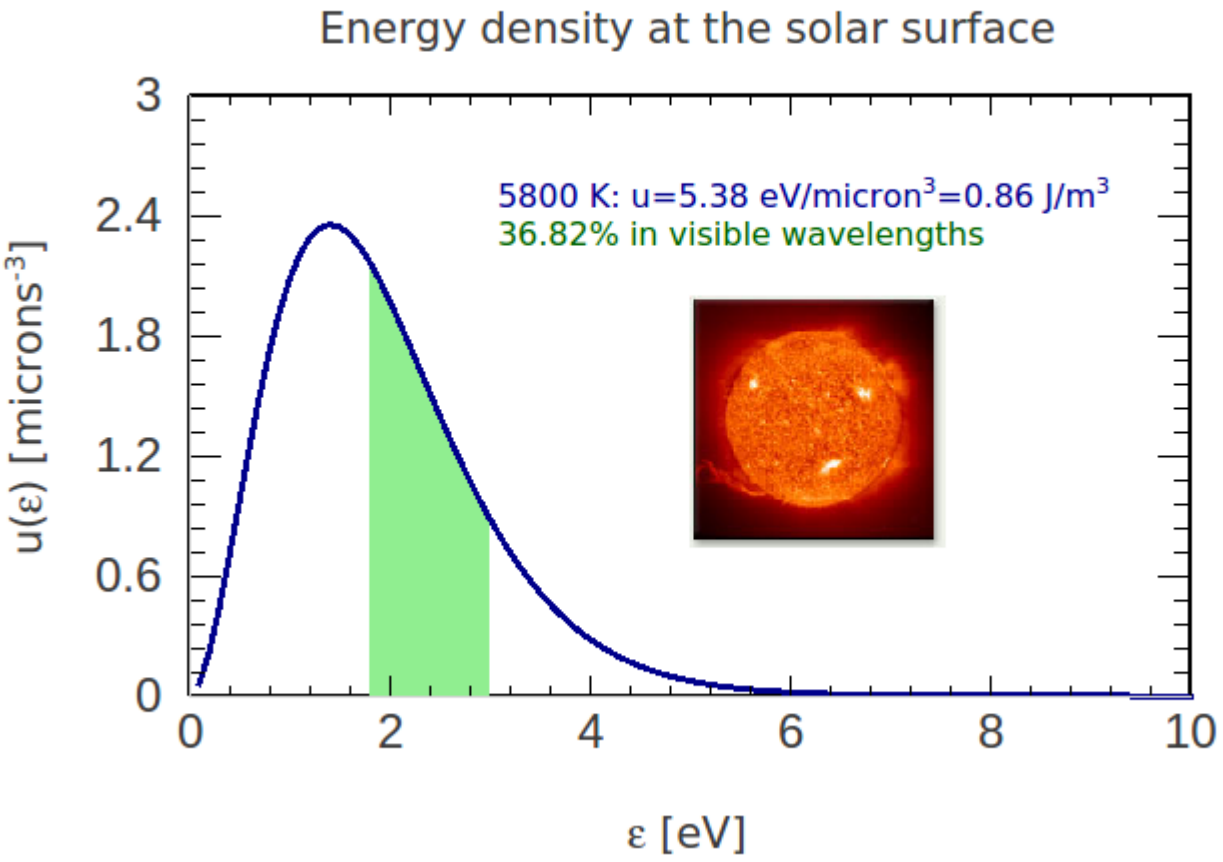


Figure 5: A PlotView displaying a `PlotFunction1D`, with manually edited axes and title. The axes may also be labelled programmatically. Since the `PlotView` inherits `QGraphicsView`, arbitrary graphics can be added to the plot.

Constructor; this form allows to specify the domain and range of the plotter (through the variable `rect`); linear or logarithmic x and y axes. If `createRangeDivider` is false, you can provide your own range divider, e.g. a custom range divider that lets you put any alphanumeric characters on the axes.

- `PlotView(PRectF rect, Style xStyle=LINEAR, Style yStyle=LINEAR, bool createRangeDividers=true, QWidget *parent=0);`

Destructor:

- `~PlotView();`

Modifiers to set the x or y range divider; mostly used to customize the axis labelling.

- `void setXRangeDivider(AbsRangeDivider * xDivider);`
- `void setYRangeDivider(AbsRangeDivider * yDivider);`

Access the bounding rectangle, in the coordinates of the plot:

- `virtual PRectF * rect();`
- `virtual const PRectF * rect() const;`

Add a `Plottable` to the `PlotView`:

- `virtual void add (Plotable *);`

Access the scene directly, so that arbitrary graphics may be added to the plot:

- `virtual QGraphicsScene* scene();`

Determine whether the statistics box is visible:

- `virtual bool isBox() const;`

Determine whether the grid is visible:

- `virtual bool isGrid() const;`

Determine whether a ruling at x=0 is visible:

- `virtual bool isXZero() const;`

Determine whether a ruling at y=0 is visible

- `virtual bool isYZero() const;`

Determine the origin (x,y) of the statistics box:

- `virtual qreal statBoxX() const;`
- `virtual qreal statBoxY() const;`

Determine whether the x- or y- axes are logarithmic:

- `virtual bool isLogX() const;`

- `virtual bool isLogY() const;`

Determine the fraction of the plot taken up by the statistics box, in x- and y-directions.

- `virtual qreal labelXSizePercentage() const;`
- `virtual qreal labelYSizePercentage() const;`

Clear the plotter:

- `virtual void clear();`

Get the text edits holding the text labels (see `QTextEdit` documentation from `Qt` library). The *vLabel* runs vertically up the right hand side of the plotter.

- `QTextEdit *titleTextEdit();`
- `QTextEdit *xLabelTextEdit();`
- `QTextEdit *yLabelTextEdit();`
- `QTextEdit *vLabelTextEdit();`
- `QTextEdit *statTextEdit();`

Get the x- and y-axis font (see `QFont` documentation from the `Qt` library)

- `QFont & xAxisFont() const;`
- `QFont & yAxisFont() const;`

Public slots Note: a *slot* is a `Qt` concept, not a C++ concept, and is not in the C++ language per se. See the `Qt` documention for more information on signals or slots, and note that a slot can always be called, just like a normal method.

Turn the grid on or off:

- `void setGrid(bool flag);`

Turn on or off a vertical ruling at x=0:

- `void setXZero(bool flag);`

Turn on or off a horizontal ruling at y=0:

- `void setYZero(bool flag);`

Turn on or off a logarithmic x-axis:

- `void setLogX(bool flag);`

Turn on or off a logarithmic y-axis:

- `void setLogY(bool flag);`

Turn on or off the visibility of the statistics box:

- `void setBox(bool flag);`

Set the x and y position of the statistics box:

- `void setLabelPos(qreal x, qreal y);`

Set the size of the statistics box in x- and y- as a percentage of the `PlotView`:

- `void setLabelXSizePercentage(qreal perCento);`
- `void setLabelYSizePercentage(qreal perCento);`

Set the domain and the range of the plot:

- `virtual void setRect(PRectF );`

Save the plot to an output file; if the file extension is .svg it will be saved in svg format, otherwise it will be stored as a pixmap.

- `void save (const std::string & filename);`

Save the plot; prompting user for a filename in a dialog:

- `void save();`

Completely redraws the `PlotView`:

- `void recreate();`

Copies the `PlotView` to the Qt clipboard:

- `void copy();`

Print the PlotView. User is prompted for the printer and configuration in a dialog:

- `void print();`

Signals: The following signal is emitted by `PlotView` when the mouse button is clicked somewhere in the plot. It returns the point (in plot coordinates) of the selected point.

- `void pointChosen(double x, double y);`

3.0.2 class AbsRangeDivider

`#include "PlotWidgets/AbsRangeDivider.h"`

Description: Abstract base class and interface for range dividers. Divides a range [minimum,maximum] into subdivisions.

3.0.3 class CustomRangeDivider

#include "QatPlotWidgets/CustomRangeDivider"

Inherits: AbsRangeDivider

Description: The custom range divider allows to subdivide an interval, and to label the subdivisions, in a custom way which is under the control of the user.

Methods: Constructor:

- `CustomRangeDivider();`

Insert a subdivision. The label is cloned & the clone managed

- `void add (double x, QTextDocument *label=NULL);`

Insert a subdivision. The label is plain text. If specified, a font is used in the text display.

- `void add(double x, const std::string & label, const QFont *font=NULL);`

Three methods to set the range. The calling the first method once is more efficient than calling the last methods twice!

- `virtual void setRange(double min, double max);`
- `virtual void setMin(double min);`
- `virtual void setMax(double max);`

3.0.4 class LinearRangeDivider

`#include "QatPlotWidgets/LinearRangeDivider.h"`

Inherits: `AbsRangeDivider`

Description: The `LinearRangeDivider` is used divide up a range into intervals of equal size. It is used to label an axis in a linear plot.

Methods: Constructor

- `LinearRangeDivider();`

Three methods to set the range. The calling the first method once is more efficient than calling the last methods twice!

- `virtual void setRange(double min, double max);`
- `virtual void setMin(double min);`
- `virtual void setMax(double max);`

3.0.5 class LogRangeDivider

`#include "QatPlotWidgets/LogRangeDivider.h"`

Inherits: `AbsRangeDivider`

Description: The `LogRangeDivider` is used divide up a range into intervals of equal size. It is used to label an axis in a linear plot.

Methods: Constructor

- `LogRangeDivider();`

Three methods to set the range. The calling the first method once is more efficient than calling the last methods twice!

- `virtual void setRange(double min, double max);`
- `virtual void setMin(double min);`
- `virtual void setMax(double max);`

3.0.6 class RangeDivision

`#include "QatPlotWidgets/RangeDivision.h"`

Description: The RangeDivision class represents a "tick mark" on an axis; it contains the location of the tick and also a text label.

Methods: Constructor:

- `RangeDivision(double value);`

Accessors:

- `const double & x() const;`
- `QTextDocument *label() const;`

Modifier (deep copy):

- `void setLabel(QTextDocument *doc);`

3.0.7 class MultipleViewWindow

`#include "QatPlotWidgets/MultipleViewWindow.h"`

Inherits: `QMainWindow` (see the `Qt` documentation).

Description: This is a convenience class that lets you put several widgets (particularly `PlotViews`) together in a tabbed Widget. It is intended for people to be able to do this without knowing that much about Qt. You can of course do the same thing much more flexibly with raw Qt–and much more.

Methods: Constructor:

- `MultipleViewWindow(QWidget *parent=0);`

Add another widget (e.g. a `PlotView`). Creates a tab and a layout for the widget and places it in the `MultipleViewWindow`.

- `void add (QWidget *w, const std::string & tabTitle);`

3.0.8 class MultipleViewWidget

`#include "QatPlotWidgets/MultipleViewWidget.h"`

Inherits: `QMainWidget` (see the `Qt` documentation).

Description: This is a convenience class that lets you put several widgets (particularly `PlotViews`) together in a tabbed Widget.

Methods: Constructor:

- `MultipleViewWidget(QWidget *parent=0);`

Add another widget (e.g. a `PlotView`). Creates a tab and a layout for the widget and places it in the `MultipleViewWindow`.

- `void add (QWidget *w, const std::string & tabTitle);`

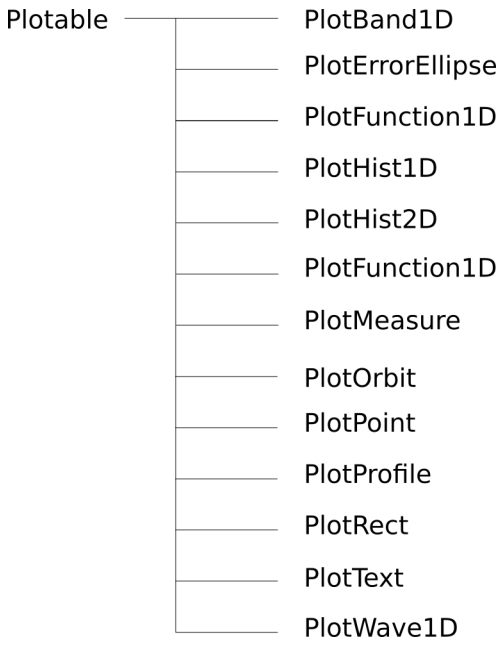


Figure 6: Class Tree Diagram for the main classes of the QatPlotting library.

4 QatPlotting Reference Manual

The `QatPlotting` library contains graphic primitives and formatting helper classes. These depend upon the Qt library , a toolkit for graphical user interfaces and two-dimensional graphics. The `PlotView` class (Section 3.0.1) extends this toolkit with a plotter, and the `QatPlotting` classes describe objects which can be placed within the plotter.

4.1 QatPlotting: classes for plot formatting

This section describes three classes: PRectF is used for determining the range and domain of a plotter; PlotStream and WPlotStream are used for formatting text labels appearing within the plot.

4.1.1 class PRectF

`#include "QatPlotWidgets/PRectF.h"`

Description: PRectF specificies the rectangular boundaries $[x_{min}, x_{max}]$, $y \in [y_{min}, y_{max}]$ for a plot.

Methods: Constructor. Creates a rectangle with $x \in [0, 1]$ and $y \in [0, 1]$

- `PRectF();`

Copy constructor (see `Qt` documentation for `QRectF`):

- `PRectF(const QRectF & rect);`

Constructor. Creates a rectangle with $x \in [x_{min}, x_{max}]$ and $y \in [y_{min}, y_{max}]$

- `PRectF`

Accessors:

- `double xmin() const`
- `double xmax() const`
- `double ymin() const`
- `double ymax() const`

Modifiers

- `void setXmin(double val)`
- `void setXmax(double val)`
- `void setYmin(double val)`
- `void setYmax(double val)`

4.1.2 class PlotStream

`#include "QatPlotting/PlotStream.h"`

Description: The `PlotStream` class is used to format text which appears in a `QTextEdit`, particularly within the `PlotView` class (see section 3.0.1) which provides access to a `QTextEdit` for the title, the x-, y-, v- axes, and the statistics box. The `PlotStream` class is used to programmatically fill these text edits with labels of various fonts, sub/superscripts, colored text, etc. Bits of text, or `PlotStream` manipulators (e.g. `PlotStream::Super`, used to force superscripts), can then be sent to the PlotStream using the left shift operator $<<$.

Methods: Constructor. Builds the `PlotStream` from a `QTextEdit`:

- `PlotStream (QTextEdit * textEdit);`

Shift Operators:

- `PlotStream & operator << (const PlotStream::Clear &);`
- `PlotStream & operator << (const PlotStream::EndP &);`
- `PlotStream & operator << (const PlotStream::Family &);`
- `PlotStream & operator << (const PlotStream::Size &);`
- `PlotStream & operator << (const PlotStream::SetPrecision &);`

- `PlotStream & operator << (const PlotStream::SetWidth &);`
- `PlotStream & operator << (const PlotStream::Super &);`
- `PlotStream & operator << (const PlotStream::Sub &);`
- `PlotStream & operator << (const PlotStream::Normal &);`
- `PlotStream & operator << (const PlotStream::Left &);`
- `PlotStream & operator << (const PlotStream::Right &);`
- `PlotStream & operator << (const PlotStream::Center &);`
- `PlotStream & operator << (const PlotStream::Bold &);`
- `PlotStream & operator << (const PlotStream::Regular &);`
- `PlotStream & operator << (const PlotStream::Italic &);`
- `PlotStream & operator << (const PlotStream::Oblique &);`
- `PlotStream & operator << (const PlotStream::Color &);`
- `PlotStream & operator << (const std::string &);`
- `PlotStream & operator << (const char);`
- `PlotStream & operator << (double);`
- `PlotStream & operator << (int);`

Public member data: Holds the text edit attached to this `PlotStream`:

- `QTextEdit *textEdit;`

Holds a text stream used for formatting of integers, characters, and double precisions. Access is provided in case application programmers have unusual formatting requirements.

- `std::ostringstream stream;`

`PlotStream` manipulators (nested classes):

- `PlotStream::Clear`
  - clears the `PlotStream`
  - constructor `PlotStream::Clear();`
- `PlotStream::EndP`
  - closes the `PlotStream` and sends all text to the `QTextEdit`
  - constructor `PlotStream::EndP();`

- `PlotStream::Family`
  - sets the font family
  - constructor `PlotStream::Family(const std::string & name);`
- `PlotStream::SetPrecision`
  - sets the precision of numeric constants directed to the `PlotStream`
  - constructor `PlotStream::SetPrecision (unsigned int val);`
- `PlotStream::SetWidth`
  - sets the width of numeric constants directed to the `PlotStream`
  - constructor `PlotStream::SetWidth (unsigned int val);`
- `PlotStream::Size`
  - sets the font size
  - constructor `PlotStream::Size (unsigned int val);`
- `PlotStream::Super`
  - sets superscript font
  - constructor `PlotStream::Super();`
- `PlotStream::Sub`
  - sets subscript font
  - constructor `PlotStream::Sub();`
- `PlotStream::Normal`
  - sets normal (neither subscript nor superscript) font
  - constructor `PlotStream::Normal();`
- `PlotStream::Bold`
  - sets boldface font
  - constructor `PlotStream::Bold();`
- `PlotStream::Regular`
  - sets regular (non-boldface) font
  - constructor `PlotStream::Regular();`
- `PlotStream::Italic`

  - sets italic font
  - constructor `PlotStream::Italic();`

- `PlotStream::Oblique`
  - sets oblique (non-italic) font
  - constructor `PlotStream::Oblique();`

- `PlotStream::Left`
  - sets left alignment
  - constructor `PlotStream::Left();`

- `PlotStream::Right`
  - sets right alignment
  - constructor `PlotStream::Right();`

- `PlotStream::Center`
  - sets center alignment
  - constructor `PlotStream::Center();`

- `PlotStream::Color`
  - sets font color
  - constructor `PlotStream::Color(const QColor & );` See `Qt` documentation on `QColor`.

- `PlotStream::Italic`
  - sets italic font
  - constructor `PlotStream::Italic();`

4.1.3 class WPlotStream

`#include "QatPlotting/WPlotStream.h"`

Description: The class `WPlotStream` is identical to `PlotStream`, except it is used in conjuntion with wide characters (`wchar_t`), wide strings (`std::wstring`) and uses an `std::wostringstream` for formatting. It is used to format text which appears in a `QTextEdit`, particularly within the `PlotView` class (see section 3.0.1) which provides access to a `QTextEdit` for the title, the x-, y-, v- axes, and the statistics box. The `WPlotStream` class is used to programmatically fill these text edits with labels of various fonts, sub/superscripts, colored text, etc. Bits of text, or `WPlotStream` manipulators (e.g. `WPlotStream::Super`, used to force superscripts), can then be sent to the WPlotStream using the left shift operator $<<$.

Methods: Constructor. Builds the `WPlotStream` from a `QTextEdit`:

- `WPlotStream (QTextEdit * textEdit);`

Shift Operators:

- `WPlotStream & operator << (const WPlotStream::Clear &);`
- `WPlotStream & operator << (const WPlotStream::EndP &);`
- `WPlotStream & operator << (const WPlotStream::Family &);`
- `WPlotStream & operator << (const WPlotStream::Size &);`
- `WPlotStream & operator << (const WPlotStream::SetPrecision &);`
- `WPlotStream & operator << (const WPlotStream::SetWidth &);`
- `WPlotStream & operator << (const WPlotStream::Super &);`
- `WPlotStream & operator << (const WPlotStream::Sub &);`
- `WPlotStream & operator << (const WPlotStream::Normal &);`
- `WPlotStream & operator << (const WPlotStream::Left &);`
- `WPlotStream & operator << (const WPlotStream::Right &);`
- `WPlotStream & operator << (const WPlotStream::Center &);`
- `WPlotStream & operator << (const WPlotStream::Bold &);`
- `WPlotStream & operator << (const WPlotStream::Regular &);`
- `WPlotStream & operator << (const WPlotStream::Italic &);`
- `WPlotStream & operator << (const WPlotStream::Oblique &);`
- `WPlotStream & operator << (const WPlotStream::Color &);`
- `WPlotStream & operator << (const std::string &);`
- `WPlotStream & operator << (const char);`
- `WPlotStream & operator << (double);`
- `WPlotStream & operator << (int);`

Public member data: Holds the text edit attached to this `WPlotStream`:

- `QTextEdit *textEdit;`

Holds a text stream used for formatting of integers, characters, and double precisions. Access is provided in case application programmers have unusual formatting requirements.

- `std::ostringstream stream;`

`WPlotStream` manipulators (nested classes):

- `WPlotStream::Clear`
  - clears the `WPlotStream`
  - constructor `WPlotStream::Clear();`
- `WPlotStream::EndP`
  - closes the `WPlotStream` and sends all text to the `QTextEdit`
  - constructor `WPlotStream::EndP();`
- `WPlotStream::Family`
  - sets the font family
  - constructor `WPlotStream::Family(const std::string & name);`
- `WPlotStream::SetPrecision`
  - sets the precision of numeric constants directed to the `WPlotStream`
  - constructor `WPlotStream::SetPrecision (unsigned int val);`
- `WPlotStream::SetWidth`
  - sets the width of numeric constants directed to the `WPlotStream`
  - constructor `WPlotStream::SetWidth (unsigned int val);`
- `WPlotStream::Size`
  - sets the font size
  - constructor `WPlotStream::Size (unsigned int val);`
- `WPlotStream::Super`
  - sets superscript font
  - constructor `WPlotStream::Super();`
- `WPlotStream::Sub`
  - sets subscript font

- constructor `WPlotStream::Sub();`

- `WPlotStream::Normal`
  - sets normal (neither subscript nor superscript) font
  - constructor `WPlotStream::Normal();`

- `WPlotStream::Bold`
  - sets boldface font
  - constructor `WPlotStream::Bold();`

- `WPlotStream::Regular`
  - sets regular (non-boldface) font
  - constructor `WPlotStream::Regular();`

- `WPlotStream::Italic`
  - sets italic font
  - constructor `WPlotStream::Italic();`

- `WPlotStream::Oblique`
  - sets oblique (non-italic) font
  - constructor `WPlotStream::Oblique();`

- `WPlotStream::Left`
  - sets left alignment
  - constructor `WPlotStream::Left();`

- `WPlotStream::Right`
  - sets right alignment
  - constructor `WPlotStream::Right();`

- `WPlotStream::Center`
  - sets center alignment
  - constructor `WPlotStream::Center();`

- `WPlotStream::Color`
  - sets font color
  - constructor `WPlotStream::Color(const QColor & );` See `Qt` documentation on `QColor`.

- `WPlotStream::Italic`
  - sets italic font
  - constructor `WPlotStream::Italic();`

4.1.4 RealArg::Eq, RealArg::Gt, and RealArg::Lt;

`#include "QatPlotting/RealArg.h"`

Description: `RealArg` is a namespace containing the classes `Eq`, `Gt` and `Lt`. These are cuts (see Section 2.1.6) which require the value of the argument to be equal to, greater than, or less than (respectively) a value which is specified on the constructor to the class. Instances may be and'ed or or'ed together:

```
const Genfun::Cut<double> & cut = RealArg::Gt(-2.0) && RealArg::Lt(2.0);
```

Classes in this namespace intended for use with with `PlotFunction1D` to restrict the range of the function.

Methods: Constructor:

- `RealArg::Eq::Eq(const double & x);`
- `RealArg::Lt::Lt(const double & x);`
- `RealArg::Gt::Gt(const double & x);`

4.2 QatPlotting: classes describing plotable objects

In this section we document a set of classes representing plottable objects such as histograms, functions, various shapes like error ellipses and shaded rectangular regions. These are designed as graphics primitives to appear within the `PlotView` (section 3.0.1). They inherit from the abstract base class `Plotable`, described immediatly below, as shown in Fig. 6. The descriptions included here are brief; we leave out the description of some of the internal-use-only methods.

In general the `Plotable` classes all contain nested properties classes which can be used to adjust the appearance of the `Plotable`. For example, `PlotFunction1D` contains the nested class `PlotFunction1D::Properties`. A typical use case is something like this:

```
PlotFunction1D p = Genfun::Sin();
{
  PlotFunction1D::Properties prop;
  prop.pen.setWidth(3);
  prop.pen.setColor("darkRed");
  p.setProperties(prop);
}
```

4.2.1 class Plotable

#include "QatPlotting/Plotable.h"

Description: Abstract base class for plotable objects.

Methods: Constructor

- `Plotable();`

Destructor

- `virtual ~Plotable();`

Get the suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const=0;`

Describe to plotter, in terms of primitives:

- `virtual void describeYourselfTo (AbsPlotter *plotter) const =0;`

4.2.2 class PlotBand1D

`#include "QatPlotting/PlotBand1D.h"`

Inherits: `Plottable`

Description: `PlotBand1D` draws a shaded region into a plotter, the region is limited by two functions over a specified domain.

Properties: Nested class `PlotBand1D::Properties` agglomerates the following member data (see `Qt` documentation)

- `QPen pen;`
- `QBrush brush;`

Methods: Constructor with two bounding functions and a suggested rectangle boundary (see the `Qt` documentation for `QPointF` and `QRectF`):

- `PlotBand1D(const Genfun::AbsFunction & function1, const Genfun::AbsFunction & function2,`
  `const QRectF & rectHint = QRectF(QPointF(-10, -10), QSizeF(20, 20)));`

Constructor with two bounding functions, a suggested rectangular boundary, and a domain restriction (see the `Qt` documentation for `QPointF` and `QRectF`):

- `PlotBand1D(const Genfun::AbsFunction & function1, const Genfun::AbsFunction & function2,`
  `const Cut<double> & domainRestriction,`
  `const QRectF & rectHint = QRectF(QPointF(-10, -10), QSizeF(20, 20)));`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.3 class PlotErrorEllipse

`#include "QatPlotting/PlotErrorEllipse.h"`

Inherits: `Plottable`

Description: `PlotErrorEllipse` draws an ellipse onto the plotter. The ellipse is frequently used to display an error matrix and is specified in terms of elements of a 2×2 real symmetric matrix.

Enums: The style determines whether the ellipse contour should be at the $\Delta\chi^2 = 1$ level (`ONEUNITCHI2`), enclose an area corresponding to one Gaussian standard deviation (`ONESIGMA`), two Gaussian standard deviations (`TWOSIGMA`), three Gaussian standard deviations (`THREESIGMA`), 90% confidence or credibility (`NINETY`), 95% confidence or credibility (`NINETYFIVE`), or 99% confidence or credibility (`NINETYNINE`).

- `enum Style { ONEUNITCHI2, ONESIGMA, TWOSIGMA, THREESIGMA, NINETY, NINETYFIVE, NINETYNINE };`

Properties: Nested class `PlotErrorEllipse::Properties` agglomerates the following member data (see `Qt` documentation)

- `QPen pen;`
- `QBrush brush;`

Methods: Constructor using the coordinates of the center (`x` ,`y`) of the center of the ellipse, the diagonal elements of the error matrix (`sx2` and `sy2`), the off-diagonal elements (`sxy`) and the ellipse style, i.e. the type of contour which is to be drawn.

- `PlotErrorEllipse(double x, double y, double sx2, double sy2, double sxy, Style style=ONESIGMA);`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.4 class PlotFunction1D

`#include "QatPlotting/PlotFunction1D.h"`

Inherits: `Plottable`

Description: `PlotFunction1D` draws a function of one variable into a plotter.

Properties: Nested class `PlotFunction1D::Properties` agglomerates the following member data (see `Qt` documentation for `QPen` and `QBrush`). The `baseline` (default value 0.0) is used when shading the function with a brush pattern; and causes the shading to occur between this baseline value and the values assumed by the function.

- `QPen pen;`
- `QBrush brush;`
- double baseline

Methods: Constructor taking a GENFUNCTION and optionally a suggested rectangular boundary (see the `Qt` documentation for `QPointF` and `QRectF`):

- ```
  PlotFunction1D(Genfun::GENFUNCTION function,
  const QRectF & rectHint = QRectF(QPointF(-10, -10), QSizeF(20, 20)));
  ```

Constructor taking a GENFUNCTION, a domain restriction, and optionally a suggested rectangular boundary (see the `Qt` documentation for `QPointF` and `QRectF`):

- ```
  PlotFunction1D(Genfun::GENFUNCTION function,
  const Cut<double> & domainRestriction,
  const QRectF & rectHint = QRectF(QPointF(-10, -10), QSizeF(20, 20)));
  ```

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.5 class PlotHist1D

`#include "QatPlotting/PlotHist1D.h"`

Inherits: `Plottable`

Description: `PlotHist1D` draws a one-dimensional histogram.

Properties: Nested class `PlotHist1D::Properties` agglomerates the following member data (see `Qt` documentation for `QPen` and `QBrush`). The `plotStyle` variable determines whether the histogram is drawn as a solid line or with symbols, the latter implying that error bars are drawn. The `symbolStyle` variable determines which of our symbol styles may be drawn, circles, squares, upward pointing triangles, downward pointing triangles, and the `symbolSize` variable determines the size of the symbol (default=5) may be set.

- `QPen pen;`
- `QBrush brush;`
- `enum PlotStyle {LINES, SYMBOLS} plotStyle;`
- `enum SymbolStyle {CIRCLE, SQUARE, TRIANGLE_U, TRIANGLE_L} symbolStyle`
- `int symbolSize;`

Methods: Constructor taking a `Hist1D`:

- `PlotHist1D(const Hist1D & h);`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.6 class PlotHist2D

`#include "QatPlotting/PlotHist2D.h"`

Inherits: `Plottable`

Description: `PlotHist2D` draws a two-dimensional histogram.

Properties: Nested class `PlotHist2D::Properties` agglomerates the following member data (see `Qt` documentation for `QPen` and `QBrush`).

- `QPen pen;`
- `QBrush brush;`

Methods: Constructor taking a `Hist2D`:

- `PlotHist2D(const Hist2D & h);`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.7 class PlotKey

`#include "QatPlotting/PlotKey.h"`

Description: `PlotKey` generates a label key for a plot. The position of the key is given in plot coordinates. Each label in the key consists of a symbol plus a labeling string. Both are specified in the call to `PlotKey::add(const Plotable * plotable, const std::string &label)`. The symbol displayed in the key is taken from the properties of the plotable object.

Methods: Constructor. Constructs and empty `PlotKey` at position (x,y) in plot coordinates.

- `PlotKey(double x, double y);`

Adds a symbol + label to the key:

- `void add(const Plotable *plotable, const std::string & label);`

Obtains the natural "containing rectangle" for the key:

- `const QRectF & rectHint() const;`

Get and set the label font:

- `QFont font() const;`
- `void setFont (const QFont & font);`

4.2.8 class PlotMeasure

`#include "QatPlotting/PlotMeasure.h"`

Inherits: `Plottable`

Description: `PlotMeasure` draws one or more points on a graph with a horizontal error bar. This is most useful for comparing measurements.

Properties: Nested class `PlotMeasure::Properties` agglomerates the following member data (see `Qt` documentation for `QPen` and `QBrush`). The `symbolStyle` variable determines which of our symbol styles may be drawn, circles, squares, upward pointing triangles, downward pointing triangles, and the `symbolSize` variable determines the size of the symbol (default=5) may be set.

- `QPen pen;`
- `QBrush brush;`
- `enum SymbolStyle {CIRCLE, SQUARE, TRIANGLE_U, TRIANGLE_L} symbolStyle;`
- `int symbolSize;`

Methods: Default constructor:

- `PlotMeasure();`

Add a point (with error bars) to the set (see the `Qt` documentation for `QPointF`):

- `void addPoint(const QPointF & point, double sizePlus, double sizeMnus);`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.9 class PlotOrbit

`#include "QatPlotting/PlotOrbit.h"`

Inherits: `Plottable`

Description: `PlotOrbit` draws a curve in the x-y plane parameterized by a variable t.

Properties: Nested class `PlotOrbit::Properties` contains a single piece of member data, a `QPen` (see `Qt` documentation).

- `QPen pen;`

Methods: Constructor taking two `GENFUNCTION`s and a minimum (`t0`) and maximum (`t1`) value for the parameter t:

- `PlotOrbit(Genfun::GENFUNCTION functionX, Genfun::GENFUNCTION functionY, double t0, double t1);`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.10 class PlotPoint

`#include "QatPlotting/PlotPoint.h"`

Inherits: `Plottable`

Description: `PlotPoint` draws a single point on a graph. Note that if you have many points, `PlotProfile` is a better choice for performance reasons than multiple `PlotPoints`.

Properties: Nested class `PlotPoint::Properties` agglomerates the following member data (see `Qt` documentation for `QPen` and `QBrush`). The `symbolStyle` variable determines which of our symbol styles may be drawn, circles, squares, upward pointing triangles, downward pointing triangles, and the `symbolSize` variable determines the size of the symbol (default=5) may be set.

- `QPen pen;`
- `QBrush brush;`
- `enum SymbolStyle {CIRCLE, SQUARE, TRIANGLE_U, TRIANGLE_L} symbolStyle;`
- `int symbolSize;`

Methods: Construct on the x and y coordinatesof the point:

- `PlotPoint(double x, double y);`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.11 class PlotProfile

`#include "QatPlotting/PlotProfile.h"`

Inherits: `Plottable`

Description: `PlotProfile` draws one or more points on a graph with a vertical error bar.

Properties: Nested class `PlotProfile::Properties` agglomerates the following member data (see `Qt` documentation for `QPen` and `QBrush`). The `symbolStyle` variable determines which of our symbol styles may be drawn, circles, squares, upward pointing triangles, downward pointing triangles, and the `symbolSize` variable determines the size of the symbol (default=5) may be set. The `errorBarSize` determines the size of the *horizontal* ridges on the vertical error bar; the `drawSymbol` flag determines whether to draw a symbol in the middle of the error bar.

- `QPen pen;`
- `QBrush brush;`
- `enum SymbolStyle {CIRCLE, SQUARE, TRIANGLE_U, TRIANGLE_L} symbolStyle;`
- `int symbolSize;`
- `int errorBarSize`
- `bool drawSymbol`

Methods: Default constructor:

- `PlotProfile();`

Add points with an optional error bar:

- `void addPoint(double x, double y, double size=0);`

Add points with an optional error bar using `QPointF` (see `Qt` documentation);

- `void addPoint(const QPointF & point, double size=0);`

Add points using `QPointF` (see `Qt` documentation) with asymmetric error bars;

- `void addPoint(const QPointF & point, double errorPlus, double errorMinus);`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.12 class PlotRect

`#include "QatPlotting/PlotRect.h"`

Inherits: `Plottable`

Description: `PlotRect` draws a shaded rectangle into a plotter.

Properties: Nested class `PlotRect::Properties` agglomerates the following member data (see `Qt` documentation)

- `QPen pen;`
- `QBrush brush;`

Methods: Construct from a `QRectF` (see `Qt` documentation):

- `PlotRect(const QRectF & );`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.13 class PlotResidual1D

`#include "QatPlotting/PlotResidual1D.h"`

Inherits: `Plottable`

Description: `PlotResidual1D`, constructed from a histogram and a function, plots the residuals of the histogram with respect to the function.

Properties: Nested class `PlotResidual1D::Properties` agglomerates the following member data (see `Qt` documentation for `QPen` and `QBrush`). The `symbolStyle` variable determines which of our symbol styles may be drawn, circles, squares, upward pointing triangles, downward pointing triangles, and the `symbolSize` variable determines the size of the symbol (default=5) may be set.

- `QPen pen;`
- `QBrush brush;`
- `enum SymbolStyle {CIRCLE, SQUARE, TRIANGLE_U, TRIANGLE_L} symbolStyle`
- `int symbolSize;`

Methods: Constructor taking a `Hist1D` and a function from which to compute the residual:

- `PlotResidual1D(const Hist1D & h, Genfun::GENFUNCTION f);`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

4.2.14 class PlotText

`#include "QatPlotting/PlotText.h"`

Inherits: Plotable

Description: Displays text in a plot.

Properties: None. The text is drawn according to the `QTextDocument` (see `Qt` Documentation)

Methods: Constructor taking an x and y text position and and optional label string (see `Qt` Documentation for `QString`)

- `PlotText(double x, double y, const QString & string=QString(""));`

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

Set the text document, which allows for many formatting options:

- `void setDocument(QTextDocument * document);`

4.2.15 class PlotWave1D

`#include "QatPlotting/PlotWave1D.h"`

Inherits: `Plottable`

Description: `PlotWave1D` takes a function of two variables an renders it as a time-dependent function of one variable. The first variable is the one to be plotted, the second variable contains the time dependence.

Properties: Nested class `PlotWave1D::Properties` agglomerates the following member data (see `Qt` documentation for `QPen` and `QBrush`). The `baseline` (default value 0.0) is used when shading the function with a brush pattern; and causes the shading to occur between this baseline value and the values assumed by the function.

- `QPen pen;`
- `QBrush brush;`
- double baseline

Methods: Constructor taking a GENFUNCTION and optionally a suggested rectangular boundary (see the `Qt` documentation for `QPointF` and `QRectF`):

- ```
  PlotWave1D(Genfun::GENFUNCTION function,
  const QRectF & rectHint = QRectF(QPointF(-10, -10), QSizeF(20, 20)));
  ```

Constructor taking a GENFUNCTION, a domain restriction, and optionally a suggested rectangular boundary (see the `Qt` documentation for `QPointF` and `QRectF`):

- ```
  PlotWave1D(Genfun::GENFUNCTION function,
  const Cut<double> & domainRestriction,
  const QRectF & rectHint = QRectF(QPointF(-10, -10), QSizeF(20, 20)));
  ```

Get suggested rectangular boundary (see `Qt` documentation for `QRectF`)

- `virtual const QRectF & rectHint() const;`

Set the properties

- `void setProperties(const Properties &properties);`

Revert to default properties:

- `void resetProperties();`

Get the properties (either default, or specific)

- `const Properties &properties () const;`

---

5 QatDataAnalysis

The QatDatAnalysis library consists of classes for histograms and tables, classes for input/output of histograms and tables, classes for client/server communcation, and miscellaneous utility classes.

5.1 QatDataAnalysis: Histograms, Tables, etcetera

5.1.1 class Attribute

`#include "QatDataAnalysis/Attribute.h"`

Description: The class `Attribute` describes a type of data stored within a table.

Enums: `enum Type { DOUBLE, FLOAT, INT, UINT, UNKNOWN};`

Methods: Constructors:

- `Attribute(const std::string & name, const std::type_info & type);`
- `Attribute(const std::string & name);`

Get the attributed name:

- `std::string & name();`
- `const std::string & name() const;`

Get the data type name, as a string:

- `std::string typeName() const;`

Get the data type as an enumerated type:

- `const Type & type() const;`

The attribute id is the position of the attribute within an `AttributeList`. Until the list is locked, this is unassigned and set to -1.

- `int & attrId();`
- `unsigned int attrId() const;`

Relational operators (lexicographical order)

- `bool operator < (const Attribute & a) const;`

Equality operator (lexicographical)

- `inline bool operator== (const Attribute & a) const;`

5.1.2 class AttributeList

#include "QatDataAnalysis/AttributeList.h"

Description: `AttributeList` is a collection of `Attribute`s. You can add to the list until you lock it. After that, adding attributes generates an exception.

Typedefs: `typedef std::vector<Attribute>::const_iterator ConstIterator;`

Methods: Constructor:

- `AttributeList();`

Add an attribute to the list:

- `void add( const std::string & name, const std::type_info & type);`

Lock the attribute list:

- `void lock();`

Iterate over the attributes;

- `ConstIterator begin () const;`
- `ConstIterator end() const;`

Random access:

- `Attribute & operator[] (size_t i) ;`
- `const Attribute & operator[] (size_t i) const;`

Size of the attribute list;

- `inline size_t size() const;`

5.1.3 class Hist1D

#include "`QatDataAnalysis/Hist1D.h`"

Description: This class describes an ordinary, one-dimensional histogram.

Nested Classes: The class `Hist1D::Clockwork` contains the internals. They are fully defined in `QatDataAnalysis/Hist1D.icc`. These internals are accessible if needed–for purposes of persistification, for example. But casual users should eschew them.

Methods: Constructors. The latter creates an anonymous histogram (name: Anonymous);

- `Hist1D(const std::string & name, size_t nBins, double min, double max);`
- `Hist1D (size_t nBins, double min, double max);`

Accumulate data:

- `void accumulate(double x, double weight=1);`

Properties of the container:

- `std::string & name();`
- `const std::string & name() const;`
- `size_t nBins() const;`
- `double min() const;`
- `double max() const;`
- `double binWidth() const;`

Properties of the bins:

- `double binUpperEdge(unsigned int i) const;`
- `double binLowerEdge(unsigned int i) const;`
- `double binCenter(unsigned int i) const;`

Stored data:

- `double bin(unsigned int i) const;`
- `double binError(unsigned int i) const;`
- `size_t overflow() const;`
- `size_t underflow() const;`
- `size_t entries() const;`

Statistical properties of the data:

- `double variance() const;`
- `double mean() const;`
- `double minContents() const;`
- `double maxContents() const;`
- `double sum() const;`

Operations:

- `Hist1D & operator = (const Hist1D & source);`
- `Hist1D & operator += (const Hist1D & source);`
- `Hist1D & operator -= (const Hist1D & source);`
- `Hist1D & operator *= (double scale);`

Clear the histogram:

- `void clear ();`

Get the internals:

- `const Clockwork *clockwork() const;`

Remake from the internals:

- `Hist1D(const Clockwork * c);`

5.1.4 class Hist1DMaker

#include "`QatDataAnalysis/Hist1DMaker.h`"

Description `Hist1DMaker` is a class that lets you make a `Hist1D` from a `Table`. It uses a function of the quantities in the `Table` to do so. See `Table::symbol()` for how to access these quantities, and Section 2, particularly Subsection 2.2.50for more detailed information on building the function. An optional second function can be used to assign a weight (based again on quantities in the `Table`) to each entry.

Methods: Constructor:

- `Hist1DMaker(Genfun::GENFUNCTION f, size_t nbins, double min, double max,`
  `const Genfun::AbsFunction * weight=NULL);`

Action:

- `Hist1D operator * (const Table & t) const;`

5.1.5 class Hist2D

#include "`QatDataAnalysis/Hist2D.h`"

Description: This class describes a two-dimensional histogram or scatterplot.

Nested Classes: The class `Hist2D::Clockwork` contains the internals. They are fully defined in `QatDataAnalysis/Hist2D.icc`. These internals are accessible if needed–for purposes of persistification, for example. But casual users should eschew them.

Methods:

Enums:

- `enum OVF { Overflow, InRange, Underflow};`

Methods: Constructors

- `Hist2D(size_t nBinsX, double minX, double maxX, size_t nBinsY, double minY, double maxY);`
- `Hist2D(const std::string & name, size_t nBinsX, double minX, double maxX, size_t nBinsY, double minY, double maxY);`

Accumulate Data:

- `void accumulate(double x, double y, double weight=1);`

Properties of the container:

- `inline std::string & name();`
- `inline const std::string & name() const;`
- `inline size_t nBinsX() const;`
- `inline size_t nBinsY() const;`
- `inline double minX() const;`
- `inline double maxX() const;`
- `inline double minY() const;`
- `inline double maxY() const;`
- `inline double binWidthX() const;`
- `inline double binWidthY() const;`

Properties of the bins:

- `inline double binUpperEdgeX(unsigned int i, unsigned int j) const;`
- `inline double binLowerEdgeX(unsigned int i, unsigned int j) const;`
- `inline double binUpperEdgeY(unsigned int i, unsigned int j) const;`
- `inline double binLowerEdgeY(unsigned int i, unsigned int j) const;`
- `inline double binCenterX(unsigned int i, unsigned int j) const;`
- `inline double binCenterY(unsigned int i, unsigned int j) const;`

Stored data:

- `inline double bin(unsigned int i, unsigned int j) const;`
- `inline double binError(unsigned int i, unsigned int j) const;`
- `inline size_t overflow(OVF a, OVF b) const;`
- `inline size_t overflow() const;`

- `inline size_t entries() const;`

Statistical properties of the data:

- `inline double varianceX() const;`
- `inline double varianceY() const;`
- `inline double varianceXY() const;`
- `inline double meanX() const;`
- `inline double meanY() const;`
- `inline double minContents() const;`
- `inline double maxContents() const;`
- `inline double sum() const;`

Operations

- `Hist2D & operator = (const Hist2D & source);`
- `Hist2D & operator += (const Hist2D & source);`
- `Hist2D & operator -= (const Hist2D & source);`
- `Hist2D & operator *= (double scale);`

Clear

- `void clear();`

For accessing bins:

- `inline size_t ii(size_t i, size_t j) const;`

Get the internals:

- `const Clockwork *clockwork() const;`

Remake the histogram from the internals:

- `Hist2D(const Clockwork * c);`

5.1.6 class Hist2DMaker

#include "`QatDataAnalysis/`"

Inherits:

Description: Hist2DMaker is a class that lets you make a Hist2D from a Table. It uses two functions of the quantities in the Table to do so. See Table::symbol() for how to access these quantities, and Section 2, particularly Subsection 2.2.50for more detailed information on building the function. An optional third function can be used to assign a weight (based again on quantities in the Table) to each entry.

Methods: Constructor:

- Hist2DMaker(Genfun::GENFUNCTION fx, size_t nbinsX, double minX, double max,
  Genfun::GENFUNCTION fy, size_t nbinsY, double minY, double maxY,
  const Genfun::AbsFunction *weight=NULL);

Action:

- Hist2D operator * (const Table & t) const;

5.1.7 class HistogramManager

#include "QatDataAnalysis/HistogramManager.h"

Description A HistogramManager is a heterogenous collection of Hist1Ds, Hist2Ds, Tables, and other HistogramManagers. These may be organized into an acyclic graph (tree).

Typedefs

- typedef std::list<Hist1D *>::iterator H1Iterator;
- typedef std::list<Hist1D *>::const_iterator H1ConstIterator;
- typedef std::list<Hist2D *>::iterator H2Iterator;
- typedef std::list<Hist2D *>::const_iterator H2ConstIterator;
- typedef std::list<Table *>::iterator TIterator;
- typedef std::list<Table *>::const_iterator TConstIterator;
- typedef std::list<HistogramManager *>::iterator DIterator;
- typedef std::list<HistogramManager *>::const_iterator DConstIterator;

Methods: Constructor:

- HistogramManager(const std::string & name="ANONYMOUS");

Get name:

- std::string & name();
- const std::string & name() const;

Iteration:

- `TIterator beginTable();`
- `TConstIterator beginTable() const;`
- `TIterator endTable();`
- `TConstIterator endTable() const;`
- `H1Iterator beginH1();`
- `H1ConstIterator beginH1() const;`
- `H1Iterator endH1();`
- `H1ConstIterator endH1() const;`
- `H2Iterator beginH2();`
- `H2ConstIterator beginH2() const;`
- `H2Iterator endH2();`
- `H2ConstIterator endH2() const;`
- `DIterator beginDir();`
- `DConstIterator beginDir() const;`
- `DIterator endDir();`
- `DConstIterator endDir() const;`

Object location:

- `const Hist1D *findHist1D(const std::string &) const;`
- `const Hist2D *findHist2D(const std::string &) const;`
- `const Table *findTable (const std::string &) const;`
- `const HistogramManager *findDir (const std::string &) const;`

Object creation:

- `Hist1D *newHist1D(const std::string &, unsigned int nBins, double min, double max);`
- `Hist1D *newHist1D(const Hist1D & source);`
- ```
  Hist2D *newHist2D(const std::string &,
  unsigned int nBinsX, double minX, double maxX,
  unsigned int nBinsY, double minY, double maxY);
  ```
- `Hist2D *newHist2D(const Hist2D & source);`
  ```

- `Table *newTable (const Table & source);`
- `HistogramManager *newDir(const std::string & name);`

Object removal:

- `void purge (const std::string & name);`
- `void rm (Hist1D *o);`
- `void rm (Hist2D *o);`
- `void rm (Table *o);`
- `void rm (HistogramManager *o);`

5.1.8 class HistSetMaker

#include "`QatDataAnalysis/HistSetMaker.h`"

Description: The `HistSetMaker` schedules one or more histograms to be created from a `Table` in a single pass. The whole set of histograms is created within an output directory. This is faster than making the individualy using `Hist1DMaker` and `Hist2DMaker` multiple times. To use this class, first schedule the histograms and then execute the list of schedule actions.

Methods: // Constructor;

- `HistSetMaker();`

Execute the list of scheduled actions. The Table t is the source of the data, the HistogramManager * manager is a pointer to the HistogramManager within which the output histograms are created.

- void exec(const Table & t, HistogramManager *manager) const;

Schedule `Hist1D` creation using a function `f` of the quantities in the `Table` to do so. See `Table::symbol()` for how to access these quantities, and Section 2, particularly Subsection 2.2.50for more detailed information on building the function. An optional second function `weight` can be used to assign a weight (based again on quantities in the `Table`) to each entry:

- `void scheduleHist1D(const std::string & name, Genfun::GENFUNCTION f,`
  `unsigned int nbins, double min, double max, const Genfun::AbsFunction *weight=NULL);`

Schedule `Hist2D` creation using a two functions `fX` and `fY` of the quantities in the `Table` to do so. See `Table::symbol()` for how to access these quantities, and Section 2, particularly Subsection 2.2.50for more detailed information on building the function. An optional third function `weight` can be used to assign a weight (based again on quantities in the `Table`) to each entry:

- `void scheduleHist2D(const std::string & name, Genfun::GENFUNCTION fX, unsigned int nbinsX, double minX,`
  `double maxX, Genfun::GENFUNCTION fY, unsigned int nbinsY, double minY, double maxY, const Genfun::AbsFuncti`
  `*weight=NULL);`

Access the number of Histograms:

- `unsigned int numH1D() const;`

- `unsigned int numH2D() const;`

Access the names of the Histograms:

- `const std::string & nameH1D(unsigned int i) const;`
- `const std::string & nameH2D(unsigned int i) const;`

5.1.9 class Projection

#include "QatDataAnalysis/Projection.h"

Description: `Projection` is an class used to project data from a `Table` into a smaller `Table`, consisting of a subset of the quantities that form the `Table`. The names of the attributes to be projected are added to the `Projection`.

Methods: Constructor:

- `Projection();`

Operation on Table

- `Table operator * (const Table & table) const;`

Add a datum

- `void add(const std::string & name);`

5.1.10 class Selection

#include "QatDataAnalysis/Selection.h"

Description The `Selection` class is for thinning a `Table` by applying a cut to each row, based upon the contents of the row. The cut is specified in the constructor. The most common cut to pass is a `TupleCut` (subclass of `Cut<Tuple>`).

Methods: Constructor:

- `Selection( const Cut<Tuple> & cut);`

Operation on Table

- `Table operator * (const Table & table) const;`

5.1.11 class Table

#include "QatDataAnalysis/Table.h"

Description: A `Table` is an object which pretends to be an array of `Tuples`. In reality, the `Tuples` may actually be disk resident, depending on how the `Table` was instantiated or otherwise obtained. If you instantiate a `Table` yourself its `Tuples` will be memory resident. If you use `HistogramManager::newTable` or read from a file using an `IODriver` (see Section 5.2.1) it may be disk-resident. The `Table` class allows to create `Tables`, store data in `Tables`, access the data, and create functions which compute derived quanties and/or cut upon the data, through the use of `Genfun::GENFUNCTIONS`.

Methods: Constructor:

- `Table(const std::string & name);`

Get the title:

- `std::string & name();`
- `const std::string & name() const;`

Getting the data into the `Table`. Sets values and builds header at same time.

- `template <typename T> void add (const std::string & name, const T & t);`
- `template <typename T> void add (const Attribute & a, const T & t);`

Capture, returning a link to the constant `Tuple`. (Note, a `TupleConstLink` is a smart pointer to a `Tuple`, which is a reference counted object. Just dereference the `TupleConstLink` to get at the `Tuple`. It is actually a `ConstLink<Tuple>` (see Section 5.4.1). We don't describe it in any more detail.).

- `virtual TupleConstLink capture();`

Get the size of the table (number of tuples):

- `virtual size_t numTuples() const;`

Getting the data out of the `Table` (Note, a `TupleConstLink` is a smart pointer to a `Tuple`, which is a reference counted object. Just dereference the `TupleConstLink` to get at the `Tuple`. It is actually a `ConstLink<Tuple>` (see Section 5.4.1)We don't describe it in any more detail.):

- `virtual TupleConstLink operator [] (size_t index) const;`

Alternate. Convenient but slow.

- `template <typename T> void read (size_t index, const std::string & aname, T & t) const;`

Print:

- `std::ostream & print (std::ostream & o= std::cout) const;`

Get the size of the table (number of attributes):

- `size_t numAttributes() const;`

Get Attributes:

- `const Attribute & attribute(unsigned int i) const;`

- `const Attribute & attribute(const std::string & name) const;`
- `AttributeListConstLink attributeList() const;`

Get the attribute symbol. For building functions of `Tuple`s, using `Genfun::GENFUNCTIONS` .

- `Genfun::Variable symbol (const std::string & name) const;`

Operations:

- `void operator += (const Table &);`

Copy the `Table` (which may be disk-resident) to a simple memory resident version:

- `Table *memorize() const;`

5.1.12 class Tuple

#include "`QatDataAnalysis/Tuple.h`"

Inherits: `RCSBase`

Description: The `Tuple` class represents a single row of a table. It can hold a variety of different data types and provides access to them. Tables build tuples and monger tables. The way in which you get data out of a data is to call one of the overloaded read methods. You can then check the status to see whether the read succeeded:

```
int value;
if (tuple->read(value,i)) {
  ..
}
```

If the data type of datum i is wrong, the read will fail. This provides a way of detecting data type, which is not really recommended for repeated accesses if speed is an If speed is an issue, the datatype should be determined (say, from the table) and the fastread method should be used.

```
int value;
switch (attribute(i).type()) {
  case Attribute::DOUBLE:    {
  tuple->fastread(value,i);    break;
}
```

And then there is a third way to access the information for very fast operations on double precision representations of data:

```
const Genfun::Argument & a = tuple->asDoublePrec();
double x = a[i];
```

Methods: Constructors:

- `Tuple(AttributeListConstLink);`
- `Tuple(AttributeListConstLink, const ValueList &);`

Access to the attributeList (where header info lives):

- `AttributeListConstLink attributeList() const;`

Print method:

- `std::ostream & print (std::ostream & o= std::cout) const;`

Check status of last operation:

- `operator bool () const;`

Read in an `int`:

- `const Tuple & read (int & i, unsigned int pos) const;`
- `void fastread (int & i, unsigned int pos) const;`

Read an `unsigned int`:

- `const Tuple & read (unsigned int & i, unsigned int pos) const;`
- `void fastread (unsigned int & i, unsigned int pos) const;`

Read a `double`:

- `const Tuple & read (double & d, unsigned int pos) const;`
- `void fastread (double & d, unsigned int pos) const;`

Read a `float`:

- `const Tuple & read (float & f, unsigned int pos) const;`
- `void fastread (float & f, unsigned int pos) const;`

Access to values:

- `ValueList & valueList();`
- `const ValueList & valueList() const;`

Access to a double precision rep of all quantities. The quantities are then cached.

- `const Genfun::Argument & asDoublePrec() const;`

Uncache. Erases the cache of values stored as double precision numbers.

- `virtual void uncache() const;`

5.1.13 class TupleCut

`#include "QatDataAnalysis/TupleCut.h"`

Inherits: `Cut<Tuple>`

Description: The `TupleCut` is a cut used to select `Tuples` from a `Table`. See also the class `Selection`. It can window on quantities in the `Tuple`, and also be used in Boolean expressions.

Enums: Greater than, less than, greater than or equal to, less than or equal to, not applicable.

- `enum Type {GT,LT,GE,LE,NA};`

Methods: Constructor. Constructs with a function and an upper and lower value:

- `TupleCut (Genfun::GENFUNCTION f, double min, double max);`

Constructor, used as in: `TupleCut cut (f, TupleCut::GT, 0)` or `TupleCut cut (f, TupleCut::LT, 0)`:

- `TupleCut (Genfun::GENFUNCTION f, Type t, double val);`

5.1.14 class Value

#include "QatDataAnalysis/Value.h"

Description: A `Value` is a class for holding data of arbitary size in an array of bytes. It is essentially for internal use.

Methods: Constructor:

- `Value(const void * value, size_t sizeInBytes);`

Return as a byte array:

- `const char * asCharStar() const;`

Zero the value:

- `void clear();`

Set the value:

- `template <typename T> void setValue (const T & t);`

5.1.15 class ValueList

#include "QatDataAnalysis/"

Description: A collection class for `Values`.

Typedefs:

- `typedef std::vector<Value>::const_iterator ConstIterator;`

Methods Constructor:

- `inline ValueList();`

Destructor:

- `inline ~ValueList();`

Add an attribute to the list:

- `inline void add(const Value & value);`

Iterate over the attributes;

- `inline ConstIterator begin () const;`
- `inline ConstIterator end() const;`

Random access:

- `inline Value & operator[] (size_t i) ;`
- `inline const Value & operator[] (size_t i) const;`

Size of the attribute list;

- `inline size_t size() const;`
- `inline void clear();`

5.2 QatDataAnalysis: Input and Output

Two input/output classes persisify `HistogramManagers` holding `Hist1Ds`, `Hist2Ds`, `Tables` and other `HistogramManagers`. These classes are `IOLoader` and `IODriver`. They provide a universal interface for input and output, as well as flexibility in determining the data format. An alternative to using these classes directly is to use the classes `HIOZeroToOne`, `HIOOneToOne`, `HIONToOne`, etc., which are described below.

5.2.1 class IODriver

#include "`QatDataAnalysis/IODriver.h`"

Description: An abstract base class for input/output drivers, which are loaded dynamically. These may include `HDF5Driver` and `RootDriver`, depending upon installation options.

Methods: Constructor:

- `IODriver();`

Creates a New Histogram manager.

- `virtual HistogramManager *newManager(const std::string & filename) const;`

Open exisiting Manager:

- `virtual const HistogramManager *openManager(const std::string & filename) cons t;`

Close the file:

- `virtual void close (const HistogramManager *mgr) const;`

Write out contents of a histogram manager (created from this driver, see `newManager`, above) to the file.:

- `virtual void write (HistogramManager *mgr) const;`

5.2.2 class IOLoader

#include "`QatDataAnalysis/`"

Description: This class opens a driver for input and output of histograms and tables. The name of the driver (i.e RootDriver, HDF5Driver) can be specficied; The search path for drivers is specified in `LD_LIBRARY_PATH`. If that is not defined, then the search path defaults to `/usr/local/lib`.

Methods: Create:

- `IOLoader`();

Get a pointer to the IO driver. Valid names are "HDF5Driver" and "RootDriver". Whether these are available depends upon installation options.

- `const IODriver *ioDriver(const std::string & name) const;`

5.3 QatDataAnalysis: Classes for Client/Server communication

The client/server classes `Socket, ClientSocket, ServerSocket` are used for establishing bidirectional communication between processes. They are adapted from the examples in Rob Tougher's article in the Linux Journal, accessible online as http://tldp.org/LDP/LG/issue74/tougher.html. That article also describes how to use them.

5.3.1 class ClientSocket

#include "`QatDataAnalysis/ClientSocket.h`"

Inherits: Socket

Description: Class for interprocess communication, used by the client program.

Methods: Constructor, taking the host name and the port number of the server with whom to connect:

- `ClientSocket ( std::string host, int port );`

Write an object or builtin datatype to the socket. Writes `sizeof(T)` bytes to the socket starting at `& T`. Best reserved for ordinary structures containing no pointers.

- `template<class T> const ClientSocket& operator << ( const T & ) const;`

Read an object or builtin datatype from the socket. Reads `sizeof(T)` bytes from the socket and writing them to the memory located at `& T`. Best reserved for ordinary structures containing no pointers.

- `template<class T> const ClientSocket& operator >> ( T & ) const;`

5.3.2 class ServerSocket

#include "`QatDataAnalysis/ServerSocket.h`"

Inherits: Socket

Description: Class for interprocess communication, used by the server program.

Methods: Constructor. Make a ServerSocket listening on a specified port:

- `ServerSocket ( int port );`

Constructor:

- `ServerSocket` ();

Write an object or builtin datatype to the socket. Writes `sizeof(T)` bytes to the socket starting at `& T`. Best reserved for ordinary structures containing no pointers.

- `template <class T> const ServerSocket& operator << ( const T & query) const;`

Read an object or builtin datatype from the socket. Reads `sizeof(T)` bytes from the socket and writing them to the memory located at `& T`. Best reserved for ordinary structures containing no pointers.

- `template <class T> const ServerSocket& operator >> ( T & query) const;`

Accept a connection request from a client. Further transactions are handled through newSocket, which is initialized after the routine is called.

- `void accept ( ServerSocket& newSocket);`

5.3.3 class Socket

#include "`QatDataAnalysis/Socket.h`"

Description: Base class for `ClientSocket` and `ServerSocket`.

Methods: Constructor:

- `Socket();`

Server initialization

- `bool create();`
- `bool bind ( const int port );`
- `bool listen() const;`
- `bool accept ( Socket& ) const;`

Client initialization

- `bool connect ( const std::string host, const int port )`

Data Transmission

- `template<class T> bool send ( const T & query ) const;`
- `template<class T> bool recv ( T & query) const;`
- `void set_non_blocking;`
- `bool is_valid() const;`

5.4 QatDataAnalysis: Miscellaneous utilities

5.4.1 template<class T> class ConstLink

#include "QatDataAnalysis/ConstLink.h"

Description: Smart links to reference-counted pointers. When the pointer is deleted it decrements the reference count of the object it points to.

Methods: Constructor

- `ConstLink();`

Copy Constructor

- `ConstLink(const ConstLink< T > &right);`

Constructor

- `ConstLink (const T *target);`

Destructor

- `virtual ~ConstLink();`

Assignment

- `ConstLink< T > & operator=(const ConstLink< T > &right);`

Equality

- `bool operator==(const ConstLink< T > &right) const;`

Inequality

- `bool operator!=(const ConstLink< T > &right) const;`

Relational operator

- `bool operator<(const ConstLink< T > &right) const;`

Relational operator

- `bool operator>(const ConstLink< T > &right) const;`

Relational operator

- `bool operator<=(const ConstLink< T > &right) const;`

Relational operator

- `bool operator>=(const ConstLink< T > &right) const;`

Dereference: (*t).method();

- `virtual const T & operator * () const;`

Dereference: t->method()

- `virtual const T * operator -> () const;`

Check pointer validity: if (t) {...}

- `operator bool () const;`

5.4.2 template <class T> class Link

#include "QatDataAnalysis/Link.h"

Inherits: `template<class T> ConstLink`

Description: Link<T> is like a ConstLink<T>, except it gives non-const access to the object to which it points.

Methods: Copy Constructor:

- `Link(const Link<T> &right);`

Promotion:

- `explicit Link(const ConstLink<T> &right);`

Construct from a pointer:

- `Link (const T *addr);`

Destructor:

- `virtual ~Link();`

Assignment:

- `Link<T> & operator = (const Link<T> & right);`

Dereferencing operations:

- `virtual T & operator * ();`
- `virtual T * operator -> ();`
- `virtual const T & operator * () const;`
- `virtual const T * operator -> () const;`

5.4.3 class RCSBase

#include "QatDataAnalysis/RCSBase.h"

Description: Base class for reference counted, squeezable objects. The reference count may be incremented or decremented by clients. When the reference count returns to zero, the object deletes itself. In addition, subclasses may implement a uncache method, which cleans a cache or "squeezes" the object.

Methods: Constructor

- `RCSBase();`

Increment reference count

- `void ref() const;`

Decrement the reference count. If it decreases to zero, object is deleted. Uncache is called upon deletion.

- `void unref() const;`

Returns the reference count

- `unsigned int refCount() const;`

This method does nothing, but subclasses have a hook which allows them to shrink themselves upon demand.

- `virtual void uncache() const;`

5.4.4 template <class T> class Query

#include "QatDataAnalysis/Query.h"

Description This class is based upon the `Fallible` class from Barton & Nackman's "Scientific and Engineering C++," Addison-Wesley, 1994. A `Query<T>` object is used to return the result of a query that can fail. The default constructor creates an invalid query, whereas the constructor taking an object of type `T` creates a valid query. The `Query<T>` can be tested for validity, and it can cast itself to a `T`. However, `Query<T>` throws an exception if an invalid query is cast. This template class is a little more clean than returning -999, which happened a lot in the past and which is still comon practice. It protects the user from blithely using the value of a failed query, by throwing an exception when that happens.

Methods: Constructor. Creates a valid query.

- `Query(const T &);`

Default constructor. Creates an invalid query.

- `Query();`

Cast operator to `T`:

- `operator T() const;`

Test Validity

- `bool isValid() const;`

5.4.5 class HIOZeroToOne

#include "QatDataAnalysis/OptParse.h"

Description: Convenience class for parsing command line options for a program taking no input histogram files and writing one output histogram file. The file format is determined by the I/O driver, which is either given as an argument to the constructor, or determined by the environment variable `QAT_IO_DRIVER`,or by default taken to be the HDF5Driver. The two options currently implemented are RootDriver (root format) or HDF5Driver (HDF5 format). The command line should minimally look like:

- `programName [-v] -o outputFile`

Public Members: The output histogram manager. No need to write or to close! This happens automatically in the destructor.

- `HistogramManager *output;`

Verbose flag:

- `bool verbose;`

Methods: Constructor:

- `HIOZeroToOne(const std::string & driver="");`

Parse the command line:

- `void optParse(int & argc, char ** & argv);`

5.4.6 class HIOOneToOne

#include "QatDataAnalysis/OptParse.h"

Description: Convenience class for parsing command line options for a program taking one input histogram files and writing one output histogram file. The file format is determined by the I/O driver, which is either given as an argument to the constructor, or determined by the environment variable `QAT_IO_DRIVER`,or by default taken to be the HDF5Driver. The two options currently implemented are RootDriver (root format) or HDF5Driver (HDF5 format). The command line should minimally look like:

- `programName [-v] -i inputFile -o outputFile`

Public Members: The output histogram manager. No need to write or to close! This happens automatically in the destructor.

- `HistogramManager *output;`

The input histogram manager:

- `const HistogramManager *input;`

Verbose flag:

- `bool verbose;`

Methods: Constructor:

- `HIOOneToOne(const std::string & driver="");`

Parse the command line:

- `void optParse(int & argc, char ** & argv);`

5.4.7 class HIOOneToZero

#include "QatDataAnalysis/OptParse.h"

Description: Convenience class for parsing command line options for a program taking one input histogram file and writing no output histogram files. The file format is determined by the I/O driver, which is either given as an argument to the constructor, or determined by the environment variable `QAT_IO_DRIVER`,or by default taken to be the HDF5Driver. The two options currently implemented are RootDriver (root format) or HDF5Driver (HDF5 format). The command line should minimally look like:

- `programName [-v] -i inputFile`

Public Members: The input histogram manager.

- `const HistogramManager *input;`

Verbose flag:

- `bool verbose;`

Methods: Constructor:

- `HIOOneToZero(const std::string & driver="");`

Parse the command line:

- `void optParse(int & argc, char ** & argv);`

5.4.8 class HIONToZero

#include "QatDataAnalysis/OptParse.h"

Description: Convenience class for parsing command line options for a program taking N input histogram files and writing no output histogram files. The file format is determined by the I/O driver, which is either given as an argument to the constructor, or determined by the environment variable `QAT_IO_DRIVER`,or by default taken to be the HDF5Driver. The two options currently implemented are RootDriver (root format) or HDF5Driver (HDF5 format). The command line should minimally look like:

- `programName [-v] inputFile1 [inputFile2] [inputFile3]...`

Public Members: The input histogram manager.

- `std::vector<const HistogramManager *> input;`

Verbose flag:

- `bool verbose;`

Methods: Constructor:

- `HIONToZero(const std::string & driver="");`

Parse the command line:

- `void optParse(int & argc, char ** & argv);`

5.4.9 class HIONToOne

#include "`QatDataAnalysis/OptParse.h`"

Description: Convenience class for parsing command line options for a program taking N input histogram files and writing one output histogram file. The file format is determined by the I/O driver, which is either given as an argument to the constructor, or determined by the environment variable `QAT_IO_DRIVER`,or by default taken to be the HDF5Driver. The two options currently implemented are RootDriver (root format) or HDF5Driver (HDF5 format). The command line should minimally look like:

- `programName [-v] inputFile1 [inputFile2] [inputFile3]...`

Public Members: The input histogram manager.

- `std::vector<const HistogramManager *> input;`

The output histogram manager:

- `HistogramManager *output;`

Verbose flag:

- `bool verbose;`

Methods: Constructor:

- `HIONToOne(const std::string & driver="");`

Parse the command line:

- `void optParse(int & argc, char ** & argv);`

5.4.10 class NumericInput

#include "QatDataAnalysis/OptParse.h"

Description: This class extracts numerical inputs from the command line and returns them as double precision numbers. If you need an integer, you will have to cast. The class allows you to declare a number of variables called whatever you like, e.g. Tom, Dick, and Harry. Then it helps to extract numerical values for those parameters from a command line like:

- `programName Tom=3 Dick=2.5 Harry=10`

Methods: Constructor:

- `NumericInput();`

Declare a parameter (name, doc, value)

- `void declare(const std::string & name, const std::string & doc, double val);`

Then override with user input:

- `void optParse(int & argc, char ** & argv);`

Print the list of parameters:

- `std::string usage() const;`

Get the value of a parameter, by name:

- `double getByName (const std::string & name) const;`

---

6 QatDataModeling

The classes in this section are used for data modeling, i.e. fitting. The QatDataModeling library depends upon the CERN `MINUIT` package (MINUIT Function Minimization and Error Analysis: Reference Manual Version 94.1 F. James 1994 CERN-D-506, CERN-D506), and therefore on the Fortran runtime library, `libgfortran`.

6.0.1 class ChiSq

#include "QatDataModeling/ChiSq.h"

Inherits: `Genfun::AbsFunctional`

Description: Functional that computes the χ^2 of a function with respect to some measured points. The measured points have an abscissa and an ordinate, the ordinate has an associated error. Points are to be added to the `ChiSq` first, then the `ChiSq` value of the function can be computed.

Methods: Constructor:

- `ChiSq();`

Add a point with an error bar:

- `void addPoint(double x, double y, double dy);`

Evaluate the χ^2 w.r.t a function:

- `double operator () (const Genfun::AbsFunction & f) const`

6.0.2 class HistChi2Functional

#include "`QatDataModeling/HistChi2Functional.h`"

Inherits: `Genfun::AbsFunctional`

Description: Functional that computes the χ^2 of a function with respect to a `Hist1D`.

Methods: Constructor. The upper and lower limits if given specify the range over which points in the histogram contribute to the χ^2.The integrate flag determines whether the function is to be integrated over the bin; otherwise it is simply evaluated at the center of the bin.

- `HistChi2Functional(const Hist1D * histogram,`
  `double lower=-1E100, double upper= 1E100, bool integrate=false);`

Evaluate χ^2 of a function w.r.t the histogram

- `virtual double operator () (const Genfun::AbsFunction & function) const;`

6.0.3 class HistLikelihoodFunctional

#include "`QatDataModeling/HistLikelihoodFunctional.h`"

Inherits: `Genfun::AbsFunctional`

Description: Functional that computes the $-2\ln\mathcal{L}$ of a function with respect to a `Hist1D`. This is used in a binned likelihood fit.

Methods: Constructor. The upper and lower limits if given specify the range over which points in the histogram contribute to the χ^2.The integrate flag determines whether the function is to be integrated over the bin; otherwise it is simply evaluated at the center of the bin.

- `HistLikelihoodFunctional(const Hist1D * histogram,`
  `double lower=-1E100, double upper= 1E100, bool integrate=false);`

Evaluate the $-2\ln\mathcal{L}$ of a function w.r.t the histogram

- `virtual double operator () (const Genfun::AbsFunction & function) const;`

6.0.4 class MinuitMinimizer

#include "QatDataModeling/MinuitMinimizer.h"

Description: An interface to MINUIT which minimizes a function with respect to parameters. Assuming that the function to be minimized is a χ^2 or a $-2\ln\mathcal{L}$, it also returns a full error matrix on the parameters. This class goes not give very fine control over MINUIT, only basic functionality.

Methods: Constructor:

- `MinuitMinimizer (bool verbose=false);`

Add a parameter. Minuit minimizes the statistic w.r.t the parameters.

- `virtual void addParameter (Genfun::Parameter * par);`

Add a statistic: a function, and a functional (e.g. Chisqaured, Likelihood)

- `void addStatistic (const Genfun::AbsFunctional * functional, const Genfun::AbsFunction * function);`

Add a statistic: a (Gaussian) constraint or similar function of parameters.

- `void addStatistic (const ObjectiveFunction *);`

Tell minuit to minimize this:

- `virtual void minimize ();`

Get the parameter value:

- `double getValue(const Genfun::Parameter *) const;`

Get the parameter error:

- `double getError(const Genfun::Parameter *ipar, const Genfun::Parameter *jpar=NULL) const;`

Get the functionValue

- `double getFunctionValue() const;`

Get the status code (0: error matrix not calculated; 1: Diagonal approximation, error matrix not accurate; 2: Full matrix, diagonal elements forced positive-definite; 3: Full accurate covariance matrix.

- `int getStatus() const;`

Get all of the values, as a vector:

- `Eigen::VectorXd getValues() const;`

Get all of the errors, as a matrix:

- `Eigen::MatrixXd getErrorMatrix() const;`

Get information on parameters:

- `unsigned int getNumParameters() const;`

Get Parameter:

- `const Genfun::Parameter *getParameter( unsigned int i) const;`

Get Parameter

- `Genfun::Parameter * getParameter(unsigned int i);`

Set the minimizer command (nominally "MINIMIZE").

- `void setMinimizeCommand(const std::string & command);`

6.0.5 class ObjectiveFunction

#include "`QatDataModeling/ObjectiveFunction.h`"

Description: Abstract base class for an object that returns a value through the function call operator. This is used to interface code to a minimizer, which minimized the value of the ObjectiveFunction. Usually, this is with respect to parameters that may be members of class derived from `ObjectiveFunction`.

Methods: Constructor

- `ObjectiveFunction();`

Function call operator. If you subclass you must override.

- `virtual double operator() () const=0;`

6.0.6 class TableLikelihoodFunction

#include "`QatDataModeling/TableLikelihoodFunctional.h`"

Inherits: `Genfun::AbsFunctional`

Description:

Methods: Constructor:

- TableLikelihoodFunctional(const Table & table);

Evaluate Likelihood of a function w.r.t the data. The function should be constructed from the table, from variables obtained using the `Table::symbol(const std::string & )` method. See Section 5.1.11.

- `virtual double operator () (const Genfun::AbsFunction & function) const;`

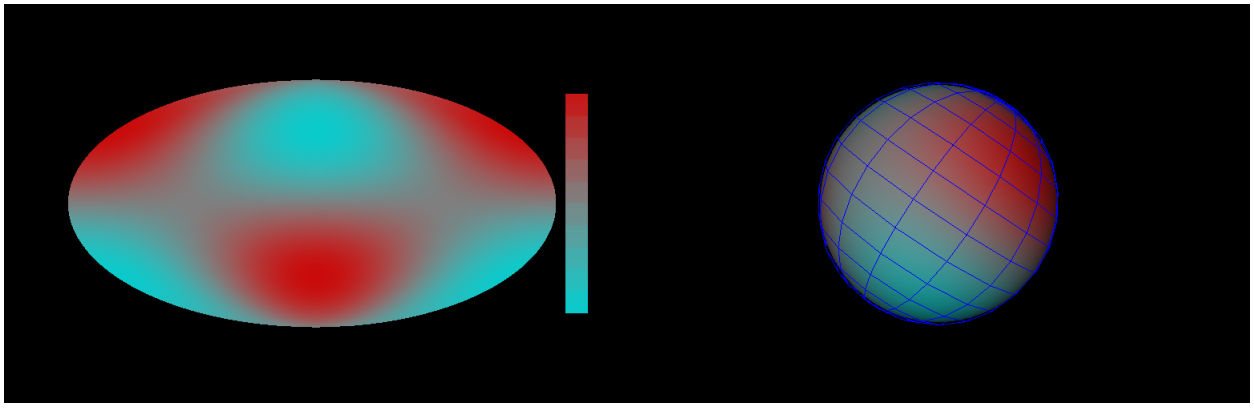


Figure 7: PolarFunctionViews. On the left, in the "unrolled" Mollweide projection; on the right, displayed on the surface of a sphere.

7 QatInventorWidgets

`QatInventorWidgets` introduces a 3D polar function plotter and currently this is the only class in the library. To use this class you will need to add

LIBS += -L/usr/local/lib -lQatInventorWidgets -lSoQt -lCoin

to the project file `qt.pro`.

7.0.1 class PolarFunctionView

`#include "QatInventorWidgets/PolarFunctionView.h"`

Inherits: `QWidget`

Description: This class provides a view for functions on a sphere. The function is a `GENFUNCTION` of two variables, the first one being $\cos\theta$ and the secon one being ϕ. It is a 3D widget that can be flattened into a Mollweide projection, so there are two viewing modes that can be toggled with the u-key, x, y, and z axes which can be toggled with the a-key, and lattitude/longitude lines which can be toggled with the l-key. The view may be saved (s-key), printed (p-key), and the "squish factor" mapping the function value onto a color value may be changed (h-key). The class is a `QWidget`, and slots allow for these changes to be made programmatically.

Methods: Constructor:

- `PolarFunctionView(QWidget *parent=0);`

Add a function which must be a function of two variables ($\cos\theta$,ϕ), and be maintained in the client code while the viewer is active.

- `void setFunction (const Genfun::AbsFunction * F);`

The `PolarFunctionView` contains an `SoQtExaminerViewer`, accessed by this method:

- `SoQtExaminerViewer *getViewer() const;`

Public slots: Save the file:

- `void save (const QString & filename);`
- `void save (const std::string & filename);`

Save, pops a dialog for interactive file selection:

- `void save();`

Print:

- `void print();`

Set the squish factor, which maps the function value onto a color.

- `void setSquish();`

Refresh the sphere, triggering re-rendering:

- `void refreshSphere();`

Toggle the display of lattitude and longitude lines:

- `void toggleLatitudeLongitude(bool flag);`

Toggle the display of x, y, z axes:

- `void toggleAxes(bool flag);`

Toggle whether the function is plotted on a sphere or unrolled into Mollweide (elliptical) projection:

- `void toggleUnroll(bool flag);`